Braille-Based IoT Home Automation for Multi-Disability Empowerment

A Project Report

Submitted to the FACULTY of ENGINEERING of

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA

In partial fulfillment of the requirements,

for the award of the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

By

N. Manikanta Raghava
(23485A0424)

M.Jahnavi
(22481A04F5)

N.Venkata Siva Prasanth
(23485A0423)

Md.Habeeb Pasha
(22481A04F8)

Under the Guidance of

Mr.V.Vittal Reddy

(Associate Professor)

Department of Electronics and Communication Engineering

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU - 521356

ANDHRA PRADESH

2025-2026

Braille-Based IoT Home Automation for Multi-Disability Empowerment

A Project Report

Submitted to the FACULTY of ENGINEERING of

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA

In partial fulfillment of the requirements,

for the award of the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

By

N. Manikanta Raghava
(23485A0424)

M.Jahnavi
(22481A04F5)

N.Venkata Siva Prasanth
(23485A0423)

Md.Habeeb Pasha
(22481A04F8)

Under the Guidance of

Mr.V.Vittal Reddy

(Associate Professor)

Department of Electronics and Communication Engineering

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU - 521356

ANDHRA PRADESH

2025-2026

Department of Electronics and Communication Engineering

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU – 521356

CERTIFICATE

This is to certify that project report entitled **"Braille-Based IoT Home Automation for Multi-Disability Empowerment"** is a bonafide record of work carried out by **N.Manikanta Raghava (23485A0424),M.Jahnavi (22481A04F5), N.Venkata Siva Prasanth (23485A0423), Md.Habeeb Pasha (22481A04F8)** under my guidance and supervision in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Electronics and Communication Engineering** of **Seshadri Rao Gudlavalleru Engineering College** Affiliated to **Jawaharlal Nehru Technological University, Kakinada.**

Mr.V.Vittal Reddy
Project guide

Dr. B. Rajasekhar
Head of the Department

Acknowledgement

We are very glad to express our deep sense of gratitude to **Mr. V. Vittal Reddy**, Associate professor, Electronics and Communication Engineering Department, for his guidance and cooperation for completing this project. We convey our heartfelt thanks to him for his inspiring assistance till the end of our project.

We convey our sincere and indebted thanks to our beloved Head of the Department **Dr. B. Rajasekhar**, for his encouragement and help for completing our project successfully.

We also extend our gratitude to our principal **Dr. B. Karuna Kumar**, for the support and for providing facilities required for the completion of our project.

We impart our heartfelt gratitude to all the Lab Technicians for helping us in all aspects related to our project.

We thank our friends and all others who rendered their help directly and indirectly to complete our project.

N. Manikanta Raghava (23485A0424)

M.Jahnavi (22481A04F5)

N.Venkata Siva Prasanth (23485A0423)

Md.Habeeb Pasha (22481A04F8)

CONTENTS

Title	Page No.
List of Figures	i
List of Tables	iii
Abstract	iv
CHAPTER 1: INTRODUCTION TO BRAILLE-BASED IOT HOME AUTOMATION	1
1.0 Introduction	1
1.1 Background	1
1.2 Aim of the Project	1
1.3 Methodology	2
1.4 Significance	4
1.5 Organization of Project	5
CHAPTER 2: LITERATURE SURVEY	6
2.0 Introduction	6
2.1 Problem Statement	6
2.2 Comparative Analysis of Existing Digital-Tactile Interfaces and Smart Home Platforms	6
2.3 Proposed System	7
CHAPTER 3: SYSTEM ARCHITECTURE AND HARDWARE COMPONENTS	8
3.0 Introduction	8
3.1 System Architecture	8
3.2 Hardware Components	9
A. ESP32-WROOM-32	9
B. MB102 Breadboard Power Supply Module	13
C. 8-Channel Active-Low Relay Board	14

Title	Page No.
D. Capacitive Fan Speed Regulator Circuit	15
E. TTP223 Capacitive Touch Sensor	16
F. MQ-2 Gas Sensor	16
G. IR Flame Sensor	17
H. SG90 Micro Servo Motor	17
I. SSD1306 0.96" OLED Display	18
J. Manual Rocker/Toggle Switches (KCD4)	19
K. Tactile Pushbuttons	20
L. Active Piezo Buzzer	20
M. Vibration Motor (ERM)	21
CHAPTER 4: IMPLEMENTATION OF THE BRAILLE BASED IOT HOME AUTOMATION	22
4.0 Introduction	22
4.1 Firmware Execution Flow	22
4.2 Braille Keypad Module	24
4.2.1 Six-Button Braille Input Array	24
4.2.1.1 Hardware Construction	24
4.2.1.2 Braille Encoding and Character Decoding Logic	25
4.2.1.3 Command Buffer and Transmission Behaviour	26
4.2.2 TTP223 Capacitive Touch Sensors	26
4.2.2.1 Hardware	26
4.2.2.2 Five-Gesture Vocabulary	27
4.2.2.3 Emergency Mode Behaviour in Firmware	27
4.2.3 Buzzer and Vibration Motor Feedback Subcircuit	28
4.2.3.1 Hardware	28
4.2.3.2 Feedback Pattern Encoding	28

Title	Page No.
4.2.4 Wi-Fi Station Link and TCP Client Connection	29
4.3 Appliance Control Module	30
4.3.1 TCP Server and Command Parsing	30
4.3.1.1 Server Initialisation and Client Management	30
4.3.1.2 Command Reception and Parsing	31
4.3.2 Room Light Control Subcircuit	32
4.3.3 Capacitor-Based Fan Speed Regulation Circuit	32
4.3.3.1 Hardware Principle	32
4.3.3.2 Relay Allocation and Capacitance Mapping	33
4.3.3.3 Firmware Implementation	34
4.3.4 Servo Motor Door Lock	34
4.3.5 Dual-Sensor Fire Detection Circuit	35
4.3.5.1 Sensor Hardware and Polarity	35
4.3.5.2 Dual-Trigger Requirement and False-Alarm detection	35
4.3.5.3 MQ-2 Warm-Up Limitation	36
4.3.6 Hardware Password Entry Circuit	36
4.3.7 Manual Override Switches	37
4.3.8 OLED Status Display	37
4.3.9 Blynk Cloud Integration and State Synchronisation	38
4.3.10 Non-Volatile State Persistence	39
CHAPTER 5: RESULTS	40
5.0 Introduction	40
5.1 Appliance Control Results	40
5.1.1 Light 1 Response for Command L1	40
5.1.2 Fan 1 response for Command F11	41

Title	Page No.
5.1.3 Fan 1 response for Command F12	41
5.2 Fire Monitoring and IoT Integration Results	42
5.2.1 Fire Monitoring Results	42
5.2.2 IoT Integration Results	43
5.2.2.1 Light 1 Status Monitoring with Command L1	43
5.2.2.2 Fan 1 Status Monitoring with Command F11	44
5.2.2.3 Fan 1 Status Monitoring with Command F12	44
5.3 Password Circuit Results	45
5.3.1 Entering Correct Password	45
5.3.2 Entering Incorrect Password	45
CHAPTER 6: CONCLUSION AND FUTURE SCOPE	46
6.0 Introduction	46
6.1 Conclusion	46
6.2 Future Scope	46
References	47
Classification of Project	48
Project Outcomes Mapped with Programme Outcomes and Programme Specific Outcomes	49
Appendix A	53
Appendix B	54

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	The Braille dot mapping used to represent English characters, symbols, and numbers	4
Figure 3.1	Architectural Block Diagram of the Braille-Based IoT Home Automation System	8
Figure 3.2	ESP32-WROOM-32 Microcontroller Module	10
Figure 3.3	ESP32 Access Point (AP) and Station (STA) Modes	11
Figure 3.4	ESP32-WROOM-32 Pinout	12
Figure 3.5	MB102 Breadboard Power Supply Module	14
Figure 3.6	8-Channel Opto-Isolated Relay Module	14
Figure 3.7	Multi-Step Capacitive Reactance Based AC Voltage Regulator	15
Figure 3.8	TTP223 Capacitive Touch Sensor Module	16
Figure 3.9	MQ-2 Gas Sensor	16
Figure 3.10	Infrared (IR) Obstacle Avoidance Sensor	17
Figure 3.11	SG90 Micro Servo Motor	18
Figure 3.12	0.96-inch OLED Display	18
Figure 3.13	DPST (Double Pole Single Throw) Rocker Switch	19
Figure 3.14	6mm Momentary Tactile Push Button	20
Figure 3.15	Active Piezoelectric Buzzer	20
Figure 3.16	2N2222 NPN Transistor Driver-based Vibration Motor	21
Figure 4.1	Firmware Execution Flow for both ESP32 Modules	22
Figure 4.2	The Braille Keypad Module prototype showing the six-button row, the two TTP223 capacitive touch pads, the buzzer, and the vibration motor mounted on perfboard.	24

Figure No.	Title	Page No.
Figure 4.3	The Appliance Control Module prototype showing the relay board, connected room lights and fans, servo-operated door lock, IR flame sensor, MQ-2 gas sensor, and OLED status display.	30
Figure 4.4	Schematic of Capacitor-Based Fan Speed Regulation Circuit	33
Figure 4.5	Password Entry Circuit using Touch Sensor and Push-Button	36
Figure 5.1	Appliance control module response for command L1	40
Figure 5.2	Hardware output showing fan1 at regulation level 1	41
Figure 5.3	Hardware output showing setting fan1 to regulation level 2	41
Figure 5.4	Stimulating fire monitoring system	42
Figure 5.5	sending Acknowledgment(ACK) signal	42
Figure 5.6	Push notification on the Blynk IoT app	43
Figure 5.7	Dashboard interface for the Braille Assistive Home IoT project, displaying the real-time status of Light 1	43
Figure 5.8	Dashboard interface for the Braille Assistive Home IoT project, displaying the real-time status of fan 1 regulation level at level 1	44
Figure 5.9	Dashboard interface for the Braille Assistive Home IoT project, displaying the real-time status of fan 1 regulation level at level 2	44
Figure 5.10	OLED display transitions showing (left) the typed password being validated as correct and (right) the resulting "ACCESS GRANTED" message.	45
Figure 5.11	OLED display transitions showing (left) the typed password being validated as incorrect and (right) the resulting "ACCESS DENIED" error message.	45
Figure A.1	QR Code for Accessing Source Code of Braille keypad module through GitHub Repository	53
Figure A.2	QR Code for Accessing Source Code of Appliance control module through GitHub Repository	53
Figure B.1	QR Code for Accessing "Braille-Based IoT Home Automation System for Visually Impaired Users using ESP32 Microcontroller" – Published in IJERT, Vol. 15, Issue 03, March 2026	54

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	ESP32-WROOM-32 Developmental Board Pin Configuration and Functionality	12
Table 4.1	Feedback Pattern Encoding — Buzzer and Vibration Motor	29
Table 4.2	Supported Command Set with Executed Actions	31
Table 4.3	Capacitor-Based Fan Speed Regulation Relay and Capacitance Mapping	33

ABSTRACT

This project presents a specialized assistive technology enabling visually impaired users to independently and securely control their domestic environment. Conventional smart home interfaces—relying on high-resolution touchscreens or voice recognition—frequently fail the blind community due to visual dependency or unreliability in noisy environments. To overcome these limitations, a tactile-first control paradigm is introduced, grounded in the universal six-dot Braille code, the global standard for tactile literacy.

The system employs a decentralized dual-module architecture using two ESP32-WROOM-32 microcontrollers. The Braille Keypad Module features a six-pushbutton array replicating a Braille cell layout, converting user-entered Braille patterns into ASCII commands. To handle overlapping patterns shared between letters (A–J) and numerals (1–0), two TTP223 capacitive touch sensors act as mode selectors—left for numeric input, right for alphabetic—while simultaneous activation of both confirms and transmits the command over a persistent TCP/IP connection.

The Appliance Control Module serves as the execution hub, managing room lighting, ceiling fan speed via a four-step capacitor-based circuit, and a secure door lock simulated by a high-torque servo motor. Safety is paramount: a dual-sensor fire detection loop combining an IR flame sensor and an MQ-2 gas sensor requires both sensors to simultaneously exceed their thresholds before triggering an alert, effectively eliminating false positives caused by sunlight or incandescent bulbs.

A multi-modal feedback system employing distinct haptic vibration and auditory beep patterns—short pulses for success, long pulses for errors, and continuous vibrations for emergencies—ensures users receive real-time confirmation at every interaction. Remote monitoring is facilitated via the Blynk IoT platform, whereby caretakers receive push notifications only after an on-site user physically acknowledges an emergency, keeping the user central to the automation loop and harmonizing traditional tactile methods with modern IoT connectivity.

Key Words: *Braille Tactile Interface, IoT Assistive Technology, Visually Impaired Home Control, ESP32 Microcontroller, Dual-Sensor Fire Detection, Multi-modal Feedback, TCP/IP Communication, Blynk Cloud Integration.*

CHAPTER 1

INTRODUCTION TO BRAILLE-BASED IOT HOME AUTOMATION

1.0 Introduction

This chapter explores the evolution of the Internet of Things (IoT) and the significant accessibility barriers that visual-centric smart home interfaces present to 295 million visually impaired individuals. It defines the project's goal to restore user autonomy by establishing a sight-independent ecosystem grounded in tactile Braille literacy. The section introduces a dual-microcontroller methodology designed to provide a resilient, portable, and responsive interaction model.

1.1 Background

The rapid evolution of the Internet of Things (IoT) has made connected home environments more accessible and inexpensive than ever before. However, a fundamental issue remains largely unaddressed: most smart home interfaces are built with the assumption that the user can see. Touchscreen dashboards, mobile applications, and visual confirmation dialogs create immediate physical barriers for the approximately 295 million people living with moderate to severe visual impairment globally.

While voice-controlled assistants are often cited as a solution, they possess documented reliability issues, particularly in acoustically difficult or noisy environments, and may struggle to accurately interpret clear diction from all users. Furthermore, existing accessible proposals often rely on screen readers that merely transfer visual dependency rather than removing it. Braille, which has been the global standard for tactile literacy since 1824, offers a reliable, sight-independent foundation for communication that is currently underutilized in the field of integrated appliance control.

1.2 Aim of the Project

The overarching goal of this research is to develop a sight-independent smart home ecosystem that bridges the gap between sophisticated IoT functionality and the practical needs of Braille-literate users. By moving away from sight-reliant digital interfaces, the project aims to restore a sense of autonomy and environmental mastery to individuals with visual impairments.

To fulfil this vision, the project is structured around two objectives:

- **objective 1: Accessible Interaction and Adaptive Control**
    - Tactile Command Translation: The system aims to implement a robust firmware logic that maps physical 6-dot Braille patterns to specific ASCII command strings, allowing for complex device operations such as fan speed regulation and light toggling without visual confirmation.
    - Closed-Loop Multi-Modal Feedback: A critical aim is to eliminate "input uncertainty" by providing instantaneous haptic vibrations and auditory beeps that confirm when a command has been received, recognized, or rejected.
    - Dual-Module Operational Resilience: By adopting a split-architecture (TCP Client-Server model), the project seeks to allow the user interface (the Braille keypad) to be portable or conveniently mounted, while isolated from the electrical noise and high-voltage risks of the appliance control module.
- **objective 2: Intelligent Safety and Remote Oversight**
    - Advanced False-Alarm Suppression: The project aims to solve the common failure point of residential IR flame sensors by requiring a synchronized trigger from both an IR detector and an (Methane Quality)MQ-2 gas sensor, ensuring emergency protocols only activate during sensible fire or gas leak events.
    - Verified Emergency Response: A core objective is to integrate a user-in-the-loop safety protocol where caretaker notifications are only dispatched via Blynk IoT after the on-site user physically acknowledges the alarm through a specific touch-sensor gesture.
    - Secure Access Management: The system aims to provide a safe method for door entry through a binary-coded password circuit, utilizing auto-relocking servos to ensure the home remains secure even if the user forgets to issue a "Close" command.

1.3 Methodology

The development of the Braille-Based IoT Home Automation System was executed through a modular engineering approach, focusing on the bifurcation of user interaction and physical hardware control. The primary architectural strategy involved a dual-microcontroller topology utilizing two ESP32-WROOM-32 boards. This separation was a critical design choice

intended to isolate the sensitive tactile input interface from the electromagnetic interference and potential system instability caused by high-voltage relay switching in the appliance module. By employing this distributed model, the Braille Keypad Module functions as a portable, low-power client, while the Appliance Control Module acts as a stationary server positioned near the electrical distribution point.

Communication between these two distinct nodes is facilitated through a persistent Transmission Control Protocol (TCP) established over a 2.4 GHz Wi-Fi link. To prioritize system reliability, the network was configured in a hybrid WIFI_AP_STA mode. This dual-layer connectivity ensures that even in the event of an external router failure, the two modules maintain a direct local Access Point connection, preserving core automation functionality and fire safety monitoring without reliance on external internet infrastructure. The data exchange protocol was designed to be lightweight, utilizing standardized ASCII command strings to minimize latency and ensure that haptic and auditory feedback are delivered to the user within milliseconds of a command's execution.

The hardware integration phase involved the synchronization of diverse sensor arrays and feedback mechanisms to create a cohesive user experience. On the input side, a six-pushbutton matrix was mapped to traditional Braille cell patterns shown in Fig 1.1, supplemented by capacitive touch sensors that serve as logic gates to distinguish between alphabetic and numeric data streams. On the control side, the methodology incorporated a capacitor-based regulation circuit to provide four distinct speeds for ceiling fans, alongside an active-low relay board configuration to ensure a "fail-off" state during power interruptions. Safety protocols were reinforced through a multi-sensor fusion logic, requiring simultaneous confirmation from both an infrared flame detector and a gas sensor to validate emergency states, thereby significantly reducing the probability of false alerts common in single-sensor systems.

The core of the user interaction logic lies in a 6-bit pattern-matching algorithm that translates physical button presses into digital text. Each of the six pushbuttons on the keypad represents a specific dot in a standard Braille cell, and when pressed, they generate a unique binary state. The firmware continuously polls these states to identify the specific "dot mapping" intended by the user. Because certain Braille patterns are identical for both letters (A–J) and numbers (1–0), the system utilizes the two TTP223 capacitive touch sensors as logic qualifiers.

When the user holds the Braille pattern, the software checks the status of the touch sensors to determine the decoding mode: activation of the left sensor flags the input as a numeric value, while the right sensor designates it as an alphabetic character. Once the character is identified, it is stored in a temporary buffer. The final command is only dispatched to the appliance module after the user performs a simultaneous short touch on both sensors, which serves as a "transmit" signal, ensuring that only intended and complete instructions are processed by the system.

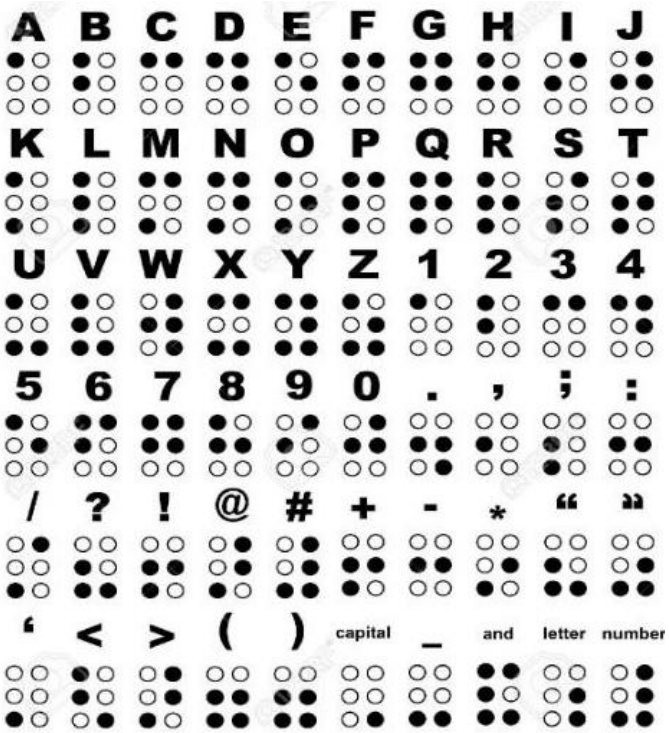

Figure 1.1: The Braille dot mapping used to represent English characters, symbols, and numbers.

1.4 Significance

The "Braille-Based IoT Home Automation System" represents a critical advancement in assistive technology by prioritizing tactile literacy over the increasingly visual-centric nature of modern smart homes. While contemporary home automation trends emphasize high-resolution touchscreens and voice-activated assistants, these interfaces inherently exclude individuals who cannot rely on sight or those who reside in acoustically challenging environments where voice recognition frequently fails. The significance of this project lies in its ability to restore a sense of environmental mastery and personal autonomy to visually impaired users by utilizing Louis Braille’s 1824 code a medium that remains the dominant global standard for tactile literacy. By building a command interface upon this established

foundation, the system ensures that users can leverage existing skills rather than having to master entirely new interaction methods.

Beyond individual empowerment, the project introduces a robust architectural framework designed for high reliability and safety. The dual-microcontroller topology is highly significant as it separates the user-facing interface from the high-voltage appliance relays, effectively isolating sensitive logic from the electromagnetic interference and noise typical of power-switching circuits. This separation also facilitates physical separation; the Braille keypad can be mounted on a wheelchair or bedside table while the control module remains at the electrical distribution point, connected via a resilient Wi-Fi TCP/IP link that functions independently of external internet stability. This ensures that local appliance control and critical fire safety monitoring remain active even during network outages.

Finally, the system addresses the critical issue of "caregiver alert fatigue" through its sophisticated emergency signaling protocol. Unlike conventional systems that dispatch automatic alerts which are often ignored if triggered by sensor noise this system implements a "human-in-the-loop" verification process. By utilizing a dual-sensor fire detection loop (IR and Gas) and requiring a physical confirmation gesture from the on-site user, the project ensures that emergency notifications sent via the Blynk cloud are highly accurate and verified. This approach not only suppresses false alarms, such as those caused by sunlight hitting an IR sensor, but also ensures that caregivers receive actionable, high-priority information when a legitimate crisis occurs.

1.5 Organization of Project

- A comparative analysis of available assistive technologies is detailed in Chapter 2, identifying where they fall short regarding visual and voice-dependency.
- A detailed breakdown of the dual-module hardware architecture and its integrated sensors is provided in Chapter 3.
- Chapter 4 describes the firmware development process, including TCP/IP protocols and Blynk IoT integration.
- The documentation of performance testing and fire detection efficiency is presented in Chapter 5.
- Chapter 6 summarizes how the dual-ESP32 architecture and tactile-first design provide a reliable, sight-independent automation system, while outlining future upgrades like a wearable Braille glove and Braille-to-voice features.

CHAPTER 2

LITERATURE SURVEY

2.0 Introduction

This chapter provides a literature survey that identifies the limitations of current market trends, which prioritize high-resolution touchscreens and voice systems over tactile security. By analysing existing digital-tactile interfaces, this section highlights gaps in hardware integration and environmental control. The chapter concludes by proposing a functional automation hub that utilizes a six-pushbutton keypad to bridge these digital-tactile disparities.

2.1 Problem Statement

Despite the rapid proliferation of smart home technologies, a significant disparity exists between the functionality of modern automation and its accessibility for the visually impaired. Current market trends heavily favour visual-centric interfaces, such as high-resolution touchscreens and complex mobile application dashboards, which create immediate physical barriers for approximately 295 million people living with moderate to severe visual impairment globally. While voice-recognition systems are often marketed as a primary solution, they remain susceptible to environmental noise, require precise diction, and often lack the tactile confirmation necessary for users to feel secure in their interactions. Furthermore, existing assistive technologies frequently function as isolated input peripherals rather than integrated systems capable of direct environmental control. There is a critical need for a cost-effective, sight-independent automation platform that provides reliable, real-time control of home appliances through established tactile literacy.

2.2 Comparative Analysis of Existing Digital-Tactile Interfaces and Smart Home Platforms

Recent research in assistive technology has explored various methods to bridge the digital-tactile gap, though many remain limited in scope or integration:

- **Vocalized Braille Input Systems**: Research presented at ICMAR 2023 described a Raspberry Pi-based Braille keyboard designed primarily for educational purposes. This system utilizes a six-key input method to vocalize Braille characters via Google Text-to-Speech (gTTS) and display them on an LCD for sighted observers. While effective for communication and learning, the system is designed as a simulation-based text-to-

speech converter and lacks the hardware interfacing required to control physical home appliances like lights or fans.

- **Scalable Remote-Control Platforms**: Recent developments in ESP32-S3 technology have introduced universal remote-control systems capable of managing multiple home devices via Wi-Fi and Bluetooth. These systems utilize MQTT protocols and web-based interfaces to offer high scalability and cost-effectiveness. However, these platforms still largely depend on smartphone apps or web browsers for interaction, which is supportive for screen readers, which do not remove the underlying visual dependency of the interface.

2.3 Proposed System

To address the limitations identified in the literature, this project proposes an integrated, dual-microcontroller system that utilizes a six-pushbutton Braille keypad as the primary control interface. Unlike existing text-to-speech peripherals, this system is a functional automation hub that converts Braille inputs directly into hardware commands over a persistent TCP/IP link.

The proposed system introduces several innovations over current models:

- **Hardware-Based Environmental Control**: By utilizing an eight-channel relay board and RC Regulator circuits, the system provides direct, multi-step control over high-voltage appliances like fans and lights.
- **Dual-Layer Safety Protocols**: Moving beyond standard single-sensor alerts, this system implements a dual-sensor fire detection loop (IR and Gas) to eliminate common false positives.
- **Human-in-the-Loop Emergency Logic**: To prevent caregiver alert fatigue, the system requires a physical acknowledgement gesture from the user before dispatching IoT notifications via the Blynk cloud.
- **Architectural Resilience**: The use of a hybrid WIFI_AP_STA mode ensures that local appliance control remains functional even during total internet or router failure, a documented weakness in many web-based smart home solutions.

CHAPTER 3

SYSTEM ARCHITECTURE AND HARDWARE COMPONENTS

3.0 Introduction

This chapter details the strategic organization of the system's hardware components within a dual-microcontroller framework. It justifies the selection of the ESP32-WROOM-32 for its internal capacitive sensing and robust wireless subsystems, which are essential for real-time assistive interaction. Additionally, this section describes the technical specifications of peripherals used in the project.

3.1 System Architecture

To provide a clear understanding of the project's physical and logical boundaries,the strategic organization of the hardware components and their integrated electrical connectivity within dual-microcontroller framework is depicted in the structural layout shown in Figure 3.1

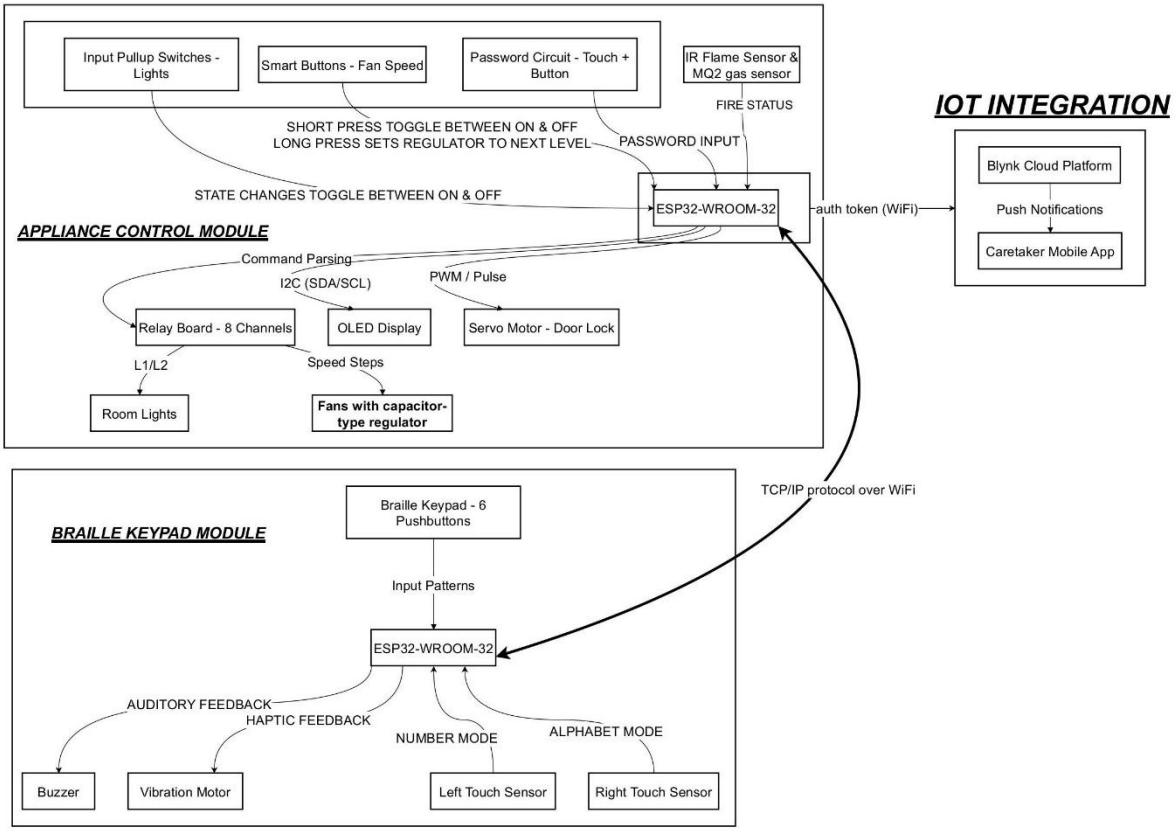

Figure 3.1: Architectural Block Diagram of the Braille-Based IoT Home Automation System

The proposed system follows a dual-module architecture that separates user interaction from appliance control, ensuring flexibility and reliability. The system consists of two ESP32-WROOM-32 controllers interconnected through a TCP/IP communication link over Wi-Fi.

At the Braille Keypad Module, a six-pushbutton interface arranged in standard Braille format captures user input as tactile patterns. These inputs are processed by the ESP32 to generate corresponding command strings. Two capacitive touch sensors enable mode selection (alphabet/number), command transmission, and emergency control. The module also integrates a buzzer and vibration motor to provide auditory and haptic feedback, ensuring accessibility for visually impaired users.

The Appliance Control Module receives commands via Wi-Fi and executes them using an eight-channel relay board. This module controls home appliances such as lights, fans with multi-level speed regulation, and a servo-based door lock. It also includes an OLED display for status indication and supports manual override through physical switches.

A dual-sensor fire detection system combining an IR flame sensor and MQ-2 gas sensor enhances safety by reducing false alarms, as alerts are triggered only when both sensors detect anomalies simultaneously.

For remote monitoring, the system integrates with the Blynk IoT Platform, enabling real-time status updates and caregiver notifications via a mobile application. Overall, this architecture ensures accessible control, reliable communication, and enhanced safety, making it suitable for assistive home automation.

3.2 Hardware Components

A. ESP32-WROOM-32

I. Technical Specifications of the ESP32-WROOM-32

The ESP32-WROOM-32 is a powerful and versatile microcontroller module designed to support a wide range of Internet of Things (IoT) and assistive technology applications. At its core, the module features the ESP32-D0WDQ6 chip, which integrates two Xtensa® 32-bit LX6 microprocessors capable of operating at adjustable clock frequencies ranging from 80 MHz to 240 MHz.

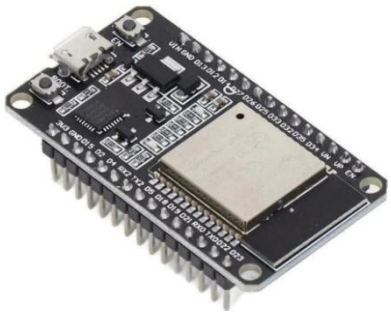

Figure 3.2: ESP32-WROOM-32 Microcontroller Module

Key architectural features include:

- **Integrated Dual-Core Processing**: The dual-core architecture allows for the efficient distribution of tasks, such as handling Wi-Fi protocols on one core while executing user-interface logic, Braille decoding, and sensor polling on the other.
- **Comprehensive Wireless Connectivity**: The module natively supports 802.11 b/g/n Wi-Fi and dual-mode Bluetooth (Classic and BLE), providing the necessary framework for persistent inter-module communication.
- **Versatile Peripheral Interface**: It includes a rich set of built-in peripherals, such as 10 capacitive touch-sensing GPIOs, Hall sensors, high-speed SPI, UART, and I2C interfaces.
- **On-Chip Memory**: The module typically includes 448 KB of ROM for booting, 520 KB of internal SRAM for data and instructions, and 4 MB of external SPI flash for firmware storage.

II. Justification for Selecting the ESP32 controller

The selection of the ESP32-WROOM-32 as the central control unit was a strategic decision based on its ability to meet the rigorous demands of a real-time assistive system. The following points justify its selection over other microcontrollers:

- **Inbuilt Wi-Fi Subsystem and Integrated Antenna**: The presence of a high-gain PCB trace antenna directly on the module ensures a reliable wireless range without requiring external hardware. This was critical for maintaining the compact and portable form factor of the Braille Keypad Module.
- **Hardware-Level Capacitive Sensing**: Unlike many other microcontrollers that require external ICs, the ESP32 features 10 internal touch-sensing GPIOs. This allowed for the

seamless implementation of alphanumeric mode switching and emergency triggers with high precision and minimal external components.

- **Dual-Role Network Topology**: The ESP32 supports simultaneous WIFI_AP_STA mode. This allows the Appliance Control Module to act as a local Access Point for a direct, persistent link to the keypad while simultaneously connecting to a home router for Blynk IoT cloud services.
- **Processing Power for Real-Time Interaction**: Its high-speed dual-core architecture ensures that Braille decoding and TCP communication occur with minimal latency (under 350 ms), allowing haptic and auditory feedback to feel instantaneous to the user.
- **Architectural Resilience**: The module's robust RF shielding and reliable performance allowed the system to run for 48 continuous hours without fault, proving its suitability for a permanent home installation.

Figure 3.3: ESP32 Access Point (AP) and Station (STA) Modes

III. General Pin Configuration and Board Architecture

The ESP32 Wroom DevKit used in this project features a 38-pin layout, providing a high density of multifunctional I/O capabilities. The image & table below describes the general functional assignment for each pin according to the hardware layout.

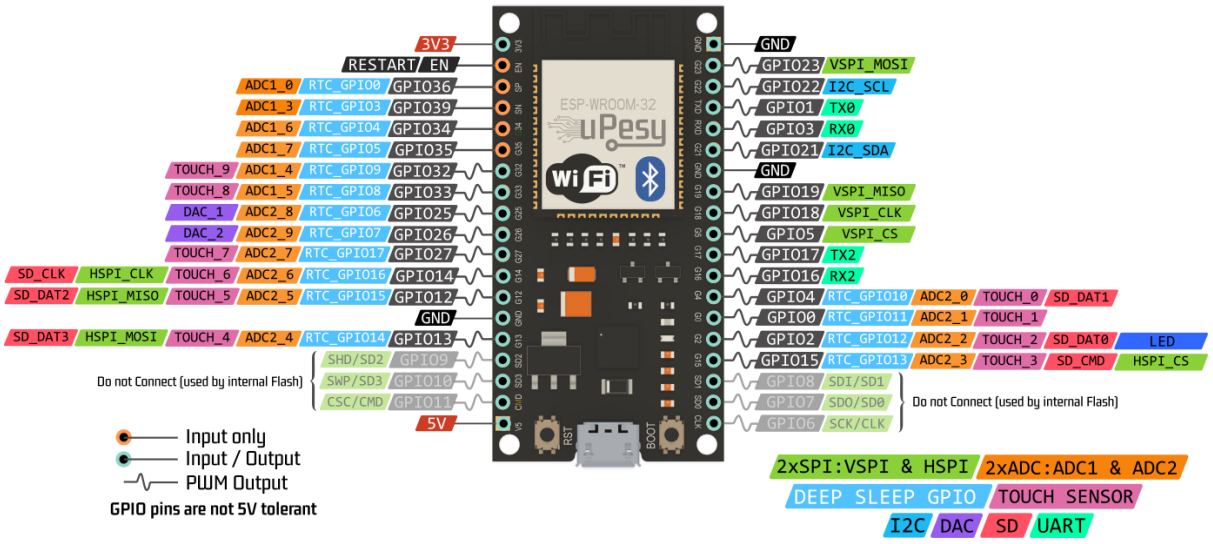

Figure 3.4: ESP32-WROOM-32 Pinout

Table 3.1: ESP32-WROOM-32 Developmental Board Pin Configuration and Functionality

Pin	Name	Primary Function Type	General Description / Functionality
1	**3V3**	Power	3.3V Power Supply
2	**EN**	Input	RESTART / Enable (Active High)
3	**SP**	Input	GPIO 36, ADC1_0, RTC_GPIO0 (Input only)
4	**SN**	Input	GPIO 39, ADC1_3, RTC_GPIO3 (Input only)
5	**G34**	Input	GPIO 34, ADC1_6, RTC_GPIO4 (Input only)
6	**G35**	Input	GPIO 35, ADC1_7, RTC_GPIO5 (Input only)
7	**G32**	I/O	GPIO 32, ADC1_4, RTC_GPIO9, TOUCH_9
8	**G33**	I/O	GPIO 33, ADC1_5, RTC_GPIO8, TOUCH_8
9	**G25**	I/O	GPIO 25, ADC2_8, RTC_GPIO6, DAC_1
10	**G26**	I/O	GPIO 26, ADC2_9, RTC_GPIO7, DAC_2
11	**G27**	I/O	GPIO 27, ADC2_7, RTC_GPIO17, TOUCH_7

12	**G14**	I/O	GPIO 14, ADC2_6, RTC_GPIO16, TOUCH_6, HSPI_CLK
13	**G12**	I/O	GPIO 12, ADC2_5, RTC_GPIO15, TOUCH_5, HSPI_MISO
14	**G13**	I/O	GPIO 13, ADC2_4, RTC_GPIO14, TOUCH_4, HSPI_MOSI
15	**GND**	Power	Ground
16	**D2**	I/O	GPIO 2, ADC2_2, RTC_GPIO12, TOUCH_2
17	**D4**	I/O	GPIO 4, ADC2_0, RTC_GPIO10, TOUCH_0
18	**RX2**	I/O	GPIO 16, UART2 RX
19	**TX2**	I/O	GPIO 17, UART2 TX
20	**G5**	I/O	GPIO 5, VSPI_CS
21	**G18**	I/O	GPIO 18, VSPI_CLK
22	**G19**	I/O	GPIO 19, VSPI_MISO
23	**G21**	I/O	GPIO 21, I2C_SDA
24	**RX0**	I/O	GPIO 3, UART0 RX
25	**TX0**	I/O	GPIO 1, UART0 TX
26	**G22**	I/O	GPIO 22, I2C_SCL
27	**G23**	I/O	GPIO 23, VSPI_MOSI
28	**GND**	Power	Ground
29-38	**-**	Mixed	Ground, 5V, and specialized Flash pins (SD2, SD3, CMD, CLK, SD0, SD1)

B. MB102 Breadboard Power Supply Module

- **Working Principle**: The MB102 module functions as a DC-to-DC step-down linear regulator station. It accepts an unregulated input voltage (typically 7V–12V DC) via a barrel jack and utilizes onboard regulators (such as the AMS1117 series) to provide two

independent, stabilized output rails. It employs filter capacitors to suppress high-frequency noise and voltage ripples, ensuring a clean power source for sensitive logic.

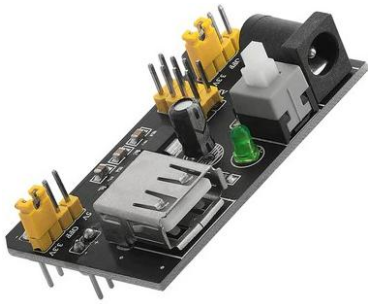

Figure 3.5: MB102 Breadboard Power Supply Module

- **Features**:
    - **Dual-Rail Flexibility**: Each output rail can be independently configured to 3.3V, 5V, or OFF via physical jumpers.
    - **Safety Indicators**: Features an onboard latching power switch and a green LED to provide immediate visual confirmation of the power state.
    - **Standardized Form Factor**: Designed to plug directly into the power rails of a standard MB102 breadboard for modular prototyping.

C. 8-Channel Active-Low Relay Board

- **Working Principle**: This board operates on the principle of electromagnetic induction and optical isolation. When the input signal is pulled "LOW," an internal optophotocoupler activates, which in turn allows current to flow through an electromagnetic coil. This coil creates a magnetic field that physically pulls a flexible armature, switching the contact from "Normally Closed" (NC) to "Normally Open" (NO), thereby completing the high-voltage AC circuit.

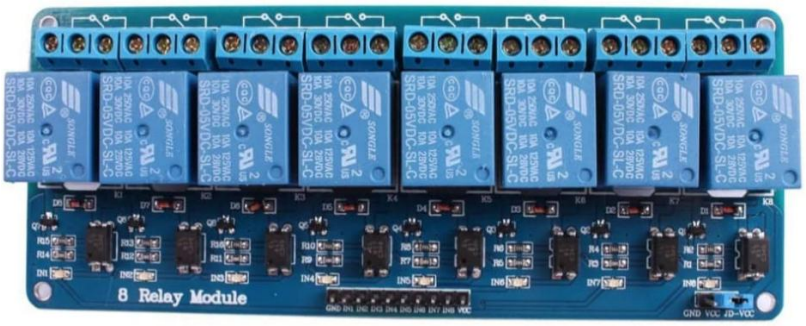

Figure 3.6: 8-Channel Opto-Isolated Relay Module

- **Features**:
    - **Opto-Isolation**: Uses PC817 optocouplers to ensure the high-voltage AC side is electrically isolated from the DC microcontroller side.
    - **JD-VCC Jumper**: Allows for the separation of coil power (5V) from signal logic (3.3V) to prevent inductive kickback.
    - **High Load Handling**: Rated to switch loads up to 10A at 250V AC or 30V DC.

D. Capacitive Fan Speed Regulator Circuit

- **Working Principle**: Unlike electronic dimmers that chop the AC waveform, this circuit operates on the principle of capacitive reactance. By placing high-voltage non-polar capacitors in series with the fan motor, the circuit increases the total impedance of the AC line, thereby reducing the current and the fan's rotational speed.

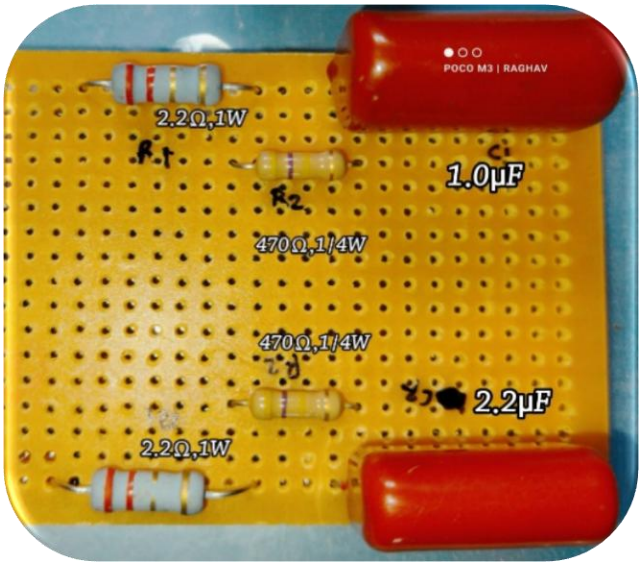

Figure 3.7: Multi-Step Capacitive Reactance based AC Voltage Regulator

- **Features**:
    - **Multiple Speed Steps**: Utilizes a combination of 1.0μF and 2.2μF capacitors along with a direct phase line to provide four distinct physical speed levels.
    - **Silent Operation**: Eliminates the "humming" or "buzzing" noise typically associated with TRIAC-based regulators.
    - **Discharge Safety**: Includes 470kΩ bleeder resistors to safely discharge stored energy when the fan is switched off.

E. TTP223 Capacitive Touch Sensor

- **Working Principle**: The TTP223 operates by detecting changes in electrical capacitance. The sensor maintains a baseline capacitance on its sensing pad; when a human finger (a conductor) approaches, it alters the dielectric constant of the local environment. An internal oscillator detects this change in the RC timing constant and outputs a digital HIGH signal.

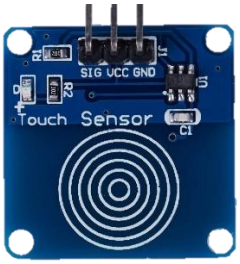

Figure 3.8: TTP223 Capacitive Touch Sensor Module

- **Features**:
  - **Zero-Force Activation**: Requires no mechanical pressure, reducing wear and tear compared to physical switches.
  - **Auto-Calibration**: Recalibrates to environmental changes in humidity and temperature every few seconds.
  - **Low Power Mode**: Consumes ultra-low current while in an idle state.

F. MQ-2 Gas Sensor

- **Working Principle**: The MQ-2 utilizes a Tin Dioxide (SnO_2) sensing layer on an alumina ceramic tube. When heated by an internal filament, the SnO_2 reacts with combustible gases (like LPG, Propane, or Smoke). In the presence of these gases, the sensor's conductivity increases, which is then translated into an analog voltage proportional to the gas concentration.

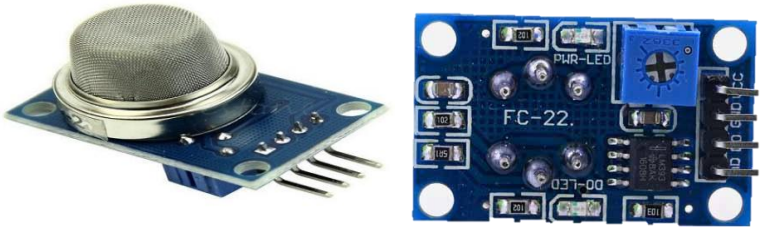

Figure 3.9: MQ-2 Gas Sensor

- **Features**:
    - **Wide Detection Range**: Sensitive to LPG, Propane, Hydrogen, Alcohol, and Smoke.
    - **Dual Output**: Provides both an Analog Output (AO) for precise concentration and a Digital Output (DO) for threshold crossing.
    - **Pre-heat Requirement**: 30-second pre-heat cycle during each boot.

G. IR Flame Sensor

- **Working Principle**: This sensor utilizes a high-speed NPN silicon photodiode that is specifically filtered to be sensitive to infrared radiation in the 760nm to 1100nm wavelength range. This range corresponds to the peak infrared emission of a typical fire's core. When IR light hits the photodiode, it generates a small current that is amplified to produce a digital signal.

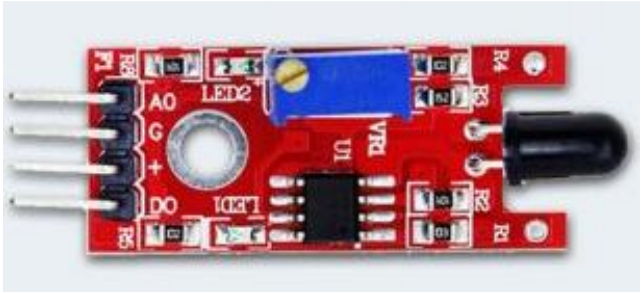

Figure 3.10: Infrared (IR) Obstacle Avoidance Sensor

- **Features**:
    - **Adjustable Sensitivity**: Features an onboard multi-turn potentiometer to calibrate the detection distance and threshold.
    - **60^0 Detection Angle**: Provides a wide field of view for monitoring large room areas.
    - **Fast Response**: Capable of detecting the presence of a flame in nanoseconds.

H. SG90 Micro Servo Motor

- **Working Principle**: The SG90 is a position-controlled actuator that operates via Pulse Width Modulation (PWM). An internal control circuit compares a 50Hz incoming PWM signal to the position of an internal potentiometer. It then drives a small DC motor through a gear train to rotate the shaft until the potentiometer matches the desired pulse width .

Figure 3.11: SG90 Micro Servo Motor

- **Features**:
    - **Precision Positioning**: Allows for specific angular control within a 180^0 arc.
    - **High Torque-to-Size**: Provides approximately 1.8 kg-cm of torque, sufficient for prototype door lock mechanisms.
    - **Lightweight Design**: Weighs only 9 grams, making it ideal for portable or small-scale enclosures.

I. SSD1306 0.96" OLED Display

- **Working Principle**: OLED (Organic Light Emitting Diode) technology uses thin films of organic molecules that emit light when an electric current is applied. Unlike LCDs, OLEDs do not require a backlight. The SSD1306 controller manages a 128/64-pixel matrix via the I2C protocol, where individual pixels are addressed to render text or graphics.

Figure 3.12: 0.96-inch OLED Display

- **Features**:
    - **I2C Communication**: Requires only two data lines (SDA and SCL) for full operation.
    - **Infinite Contrast**: Deep blacks are achieved by turning pixels completely OFF, resulting in high readability.

- **Low Power Consumption**: Extremely efficient as only the active pixels consume power.

J. Manual Rocker/Toggle Switches (KCD4)

- **Working Principle:** The KCD4 is a heavy-duty, Double Pole Single Throw (DPST) rocker switch that operates through a mechanical "snap-action" mechanism. When the rocker is pressed, an internal spring-loaded contact arm bridges the stationary terminals, creating a low-resistance path for the current. In this project, these switches are integrated into the Appliance Module as a manual override and are interfaced with the ESP32 using internal pull-up resistor logic. One terminal of the switch is connected to the ESP32 GPIO, and the other is connected to Ground (GND). When the switch is open, the internal pull-up resistor holds the GPIO at a logic HIGH state; when the switch is closed, it shorts the pin to GND, pulling the logic to a LOW state. This allows the firmware to detect a manual user intervention and toggle the corresponding relay state accordingly.

Figure 3.13: DPST (Double Pole Single Throw) Rocker Switch

- **Features:**
  - **High Power Rating:** Specifically designed for AC mains, with ratings of 16A at 250 V AC and 20 A at 125V AC.
  - **Integrated Visual Indicator:** Features a red translucent rocker that houses an internal neon lamp or LED to indicate the energized state of the circuit.
  - **Tactile Reliability:** Provides a clear, audible "click" and a distinct physical orientation, which is essential for providing feedback to visually impaired users during manual operation.

K. Tactile Pushbuttons

- **Working Principle:** A tactile pushbutton is a momentary SPST (Single Pole Single Throw) switch that facilitates a temporary electrical connection only while it is physically de-pressed. Internally, it consists of a flexible metallic dome or contact plate positioned above two stationary terminals. When the button is pressed, the dome deforms and touches both terminals, completing the circuit.

Figure 3.14: 6mm Momentary Tactile Push Button

- **Features:**
  - **Momentary Action:** The circuit automatically breaks once the pressure is released, allowing for rapid-fire data entry and distinct character separation.
  - **Compact Footprint:** Their small size (6mm x 6 mm) allows six buttons to be clustered tightly to mimic the ergonomic layout of a standard Braille cell.
  - **High Lifecycle Rating:** Designed to withstand thousands of actuations, ensuring the longevity of the portable remote unit.

L. Active Piezo Buzzer

- **Working Principle**: The active piezo buzzer utilizes the **piezoelectric effect** to generate sound. It features an internal oscillator that produces a fixed-frequency square wave when powered by a continuous DC voltage. This signal causes a piezoelectric ceramic disk to vibrate against a metal diaphragm, creating sound through mechanical displacement.

Figure 3.15: Active Piezoelectric Buzzer

- **Features**:
    - **Integrated Driver**: Unlike passive buzzers, this active version does not require an external PWM signal, simplifying the firmware logic for status alerts.
    - **High Decibel Output**: Provides a clear, penetrating sound that is easily audible in a standard household environment.
    - **Low Power Logic Drive**: Designed to operate effectively at the 3.3V logic level provided by the ESP32 GPIO pins.

M. Vibration Motor (ERM)

- **Working Principle**: This system uses an Eccentric Rotating Mass (ERM) motor to convert electrical energy into tactile vibration. A small DC motor spins an off-center weight, creating centrifugal force that causes the housing to oscillate.Because the motor's current draw exceeds the ESP32 GPIO limits, a 2N2222 NPN transistor acts as a switch. The ESP32 triggers the transistor via a 1.0kΩ resistor, allowing current to flow from the 3.3V rail through the motor. A 1N4007 flyback diode is connected in parallel to suppress inductive voltage spikes (back-EMF) and protect the circuitry.

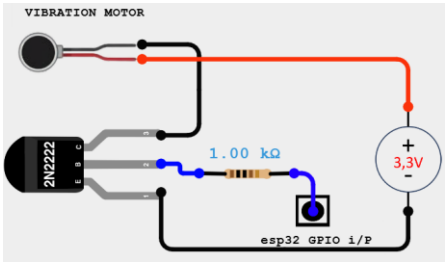

Figure 3.16: 2N2222 NPN transistor driver-based Vibration Motor

- **Features**:
- **Tactile Feedback:** Enables "silent" communication through vibration patterns (e.g., single pulse for numbers, double for letters).
- **Circuit Safety:** The transistor interface isolates the ESP32 from high current demands.
- **Inductive Protection:** The 1N4007 diode prevents damage from voltage surges when the motor stops.

CHAPTER 4

IMPLEMENTATION OF THE BRAILLE BASED IOT HOME AUTOMATION

4.0 Introduction

This chapter explains the firmware execution flow, focusing on the non-blocking loop architecture that manages Braille decoding and TCP/IP communication. It details the pattern-matching algorithm used to translate 6-bit button inputs into ASCII commands through a dedicated lookup table. Furthermore, it describes the software stack, including the use of the Arduino IDE for developing the independently compiled sketches and the Blynk IoT platform for cloud-based remote monitoring.

4.1 Firmware Execution Flow

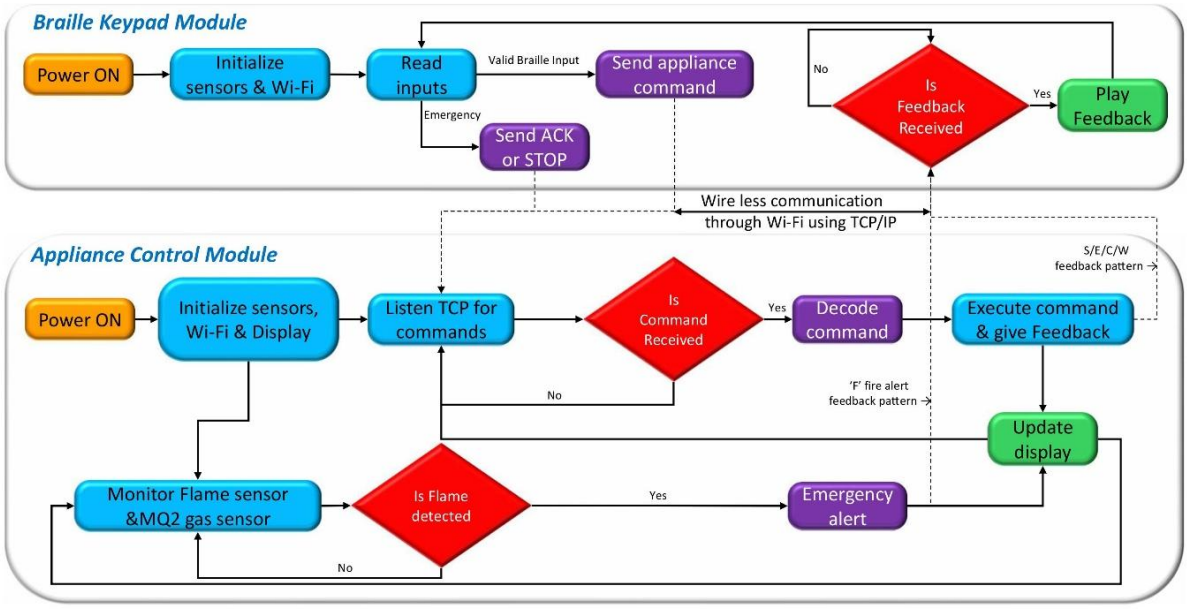

Figure 4.1: Firmware execution flow for both ESP32 modules, showing Braille input decoding, command dispatch, fire-alert handling, and the TCP feedback loop.

The system firmware is split across two independently compiled Arduino sketches one for the Braille Keypad Module and one for the Appliance Control Module but both follow the same cooperative, non-blocking loop architecture. On power-on, each module runs its setup() routine, which initialises GPIO pins, establishes the Wi-Fi link, configures peripherals, and restores any persisted state. After setup() returns, both modules enter their respective loop() functions, which execute repeatedly and indefinitely. Because no FreeRTOS task scheduler is

used, all timing decisions alert intervals, door timers, debounce windows, hold-duration checks are implemented using millis()-based timestamp comparisons rather than blocking delays, so the loop remains responsive to all concurrent inputs on every pass.

The Keypad Module loop executes in three strictly prioritised stages on every pass. The first stage calls maintainConnection() to verify that both the Wi-Fi station link and the TCP socket to the Appliance Module are live, re-establishing either if necessary, before doing anything else. The second stage reads any byte waiting in the TCP receive buffer from the Appliance Module. If the byte is 'F', the module immediately sets its emergency Mode flag and starts the continuous alert beep cycle; any other byte in the set {S, E, C, W} is forwarded to triggerFeedback() to produce the appropriate audio-haptic pattern. The third stage only executes if emergencyMode is false; it handles all normal Braille input dual-touch command transmission, single-touch character entry, and mode selection. This priority ordering ensures that a fire alert from the Appliance Module takes complete control of the keypad and suspends all normal input processing until the user explicitly resolves it.

The Appliance Module loop executes in five concurrent passes on every iteration. It begins with Blynk.run() to service the cloud heartbeat and virtual-pin synchronisation. It then checks the door auto-close timer, accepting a new TCP client connection or reading commands from the active client. After command processing, it polls the manual override switches and password circuit. Finally, it reads both fire sensors and, if both simultaneously exceed their thresholds, begins sending the 'F' alert byte to the Keypad Module at one-second intervals. The fire detection check runs on every single loop pass it is never deferred or skipped so the latency between a simultaneous dual-sensor trigger and the first alert byte reaching the keypad is bounded only by the TCP round-trip time and one loop-pass delay, which together produced sub-500 ms alert dispatch in every trial.

The two modules communicate over a persistent TCP connection on port 7890. The Appliance Module runs a WiFiServer on that port and accepts exactly one client at a time; any second connection attempt during an active session is immediately rejected. The Keypad Module is the TCP client: it transmits newline-terminated ASCII command strings upstream to the Appliance Module, and the Appliance Module responds with a single status byte 'S' for success, 'E' for unrecognised command, 'C' for correct password, 'W' for wrong password, or 'F' for active fire alert. This asymmetric exchange pattern means the feedback byte is the only downstream data the keypad ever receives, keeping the protocol minimal and the timing predictable. Commands reached the relay board in under 350 ms under stable home Wi-Fi

conditions, with the TCP round-trip accounting for only 20–50 ms of that total; relay actuation and firmware polling constituted the remainder.

4.2 Braille Keypad Module

The Braille Keypad Module is the entire user-facing half of the system. It is built around a single ESP32-WROOM-32 operating as a Wi-Fi station that connects to the access point broadcast by the Appliance Control Module. Its responsibilities are to read the user's Braille input, decode it into ASCII command strings, transmit those commands over TCP, receive feedback bytes in return, and convert those bytes into audio and tactile signals that a completely sighted-free user can interpret. Every component on this board exists to close that loop between the user's fingers and the relay board across the room.

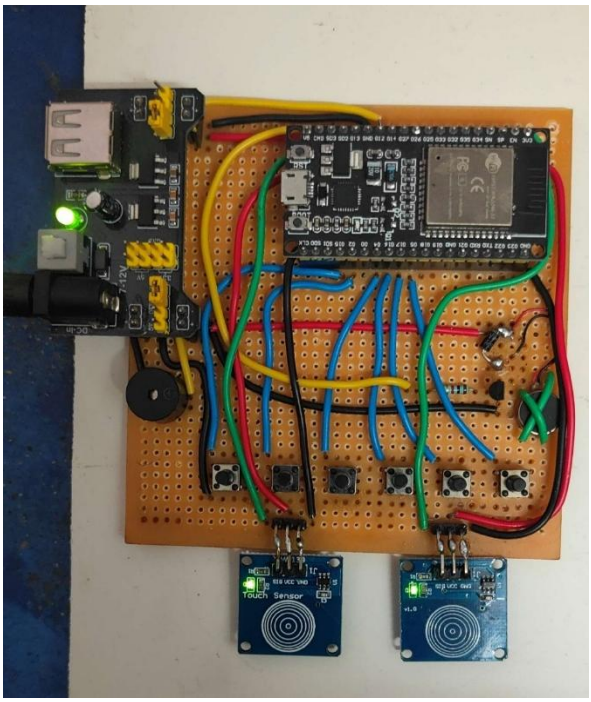

Figure 4.2: The Braille Keypad Module prototype showing the six-button row, the two TTP223 capacitive touch pads, the buzzer, and the vibration motor mounted on perfboard.

4.2.1 Six-Button Braille Input Array

4.2.1.1 Hardware Construction

Six momentary normally-open pushbuttons are arranged in a single row and wired to GPIO pins 15, 2, 4, 16, 17, and 5, mapping to Braille dots 1 through 6 from left to right respectively. The physical layout directly mirrors the standard six-dot Braille cell orientation so that any trained Braille reader can orient correctly by touch alone in seconds without instruction. Each button pin is configured as INPUT_PULLUP in firmware the ESP32's

internal pull-up resistors pull each line to 3.3 V (logic HIGH) when the button is unpressed, and pressing the button connects the pin to ground (logic LOW). This arrangement requires no external resistors and keeps the perfboard circuit clean. No hardware RC debounce network is used; all debouncing is handled entirely in the firmware, which allows the timing threshold to be adjusted per-user by editing a single constant rather than resoldering a capacitor.

4.2.1.2 Braille Encoding and Character Decoding Logic

The intellectual core of the keypad module is a 26-element byte lookup table " brailleMap[] " that encodes the standard six-dot Braille patterns for the letters A through Z. Each entry is a decimal integer whose binary representation describes which of the six Braille dots are raised. Bit 0 (value 1) corresponds to dot 1, bit 1 (value 2) to dot 2, bit 2 (value 4) to dot 3, bit 3 (value 8) to dot 4, bit 4 (value 16) to dot 5, and bit 5 (value 32) to dot 6. The letter A, which has only dot 1 raised, maps to binary 000001, decimal 1. The letter B, which has dots 1 and 2 raised, maps to binary 000011, decimal 3. The letter C, which has dots 1 and 4 raised, maps to binary 001001, decimal 9. This pattern continues for all 26 letters.

The full table in the firmware is declared as: brailleMap[] = {1, 3, 9, 25, 17, 11, 27, 19, 10, 26, 5, 7, 13, 29, 21, 15, 31, 23, 14, 30, 37, 39, 58, 45, 61, 53}, with each position in the array corresponding to the alphabetical index of the letter index 0 is A, index 1 is B, and so on up to index 25 for Z. When a touch event is detected, the firmware reads all six button GPIO states in a single for-loop pass and builds the pattern byte using pattern |= (1 << i) for each button index i whose pin reads LOW. This produces a compact six-bit integer describing exactly which dots are currently depressed. A second for-loop then iterates through all 26 entries of brailleMap; when it finds an entry equal to the measured pattern at index i, it resolves the character from brailleChars[i] a companion array of the 26 capital letters in alphabetical order.

The distinction between letter mode and number mode is determined by which touch sensor the user holds at the moment the buttons are pressed. If the right touch sensor (touchRight, GPIO 33) is active, the resolved index maps to a letter character. If the left touch sensor (touchLeft, GPIO 32) is active, the same button pattern maps to a digit: brailleMap indices 0 through 8 map to digit characters '1' through '9', and index 9 maps to '0'. This mirrors the Braille Number Indicator convention that existing Braille readers already know shifting to number mode is a context change signalled by which hand the user uses, not a separate physical action, so no new motor skill is required. Any dot pattern that does not match any of

the 26 known entries resolves to the placeholder '?' and is silently discarded without being appended to the buffer, preventing corrupted commands.

A 400 ms software debounce guard controlled by the lastDebounce timestamp prevents a single physical press from registering multiple times. The guard checks that at least 400 ms have elapsed since the last accepted character event before another press is considered. When a valid character is resolved and appended to localBuffer, a brief 100 ms simultaneous buzzer and vibration pulse fires immediately, confirming to the user that the character was accepted without any visual feedback needed. The buffer accumulates characters silently until the user triggers the transmission gesture, at which point the complete command string is sent to the Appliance Module as a newline-terminated ASCII string over TCP.

4.2.1.3 Command Buffer and Transmission Behaviour

The localBuffer string variable holds the growing command as the user types. Characters accumulate in order L then 1 becomes "L1", F then 1 then 4 becomes "F14" with each character added after resolving the current dot pattern. When the user performs the transmission gesture (both touch sensors held simultaneously for 500 ms, described in Section 2.2), the firmware calls tcpClient.print(localBuffer + "\n"), sends a newline-terminated string, and immediately clears localBuffer to an empty string. If the TCP socket is not connected at the moment of transmission, the firmware fires the error feedback pattern (800 ms pulse) and discards the buffer without sending. If the buffer is empty when the transmission gesture is detected, the error pattern also fires because there is no command to send. After transmission, the module waits up to 1000 ms for the response byte from the Appliance Module, which is processed in the first stage of the next loop pass.

4.2.2 TTP223 Capacitive Touch Sensors Mode, Transmission and Emergency Control

4.2.2.1 Hardware

Two TTP223 single-channel capacitive touch sensor modules are mounted on the perfboard, one designated touchLeft on GPIO 32 and one designated touchRight on GPIO 33. Each TTP223 outputs a stable digital HIGH when its sensing pad is touched and returns to LOW when released. Both sensor output pins are connected directly to the ESP32 GPIO inputs without external pull-down resistors because the TTP223 module includes onboard circuitry that holds the output LOW during the un-touched state. In firmware, both pins are configured as INPUT and read using digitalRead(), returning HIGH on contact and LOW on release.

4.2.2.2 Five-Gesture Vocabulary

Two physical components encode five completely distinct interaction types, making the touch sensors the most behaviourally dense part of the keypad module. Holding only the right sensor while pressing the six buttons selects letter-decoding mode. Holding only the left sensor selects number-decoding mode. Pressing both sensors simultaneously and releasing them within 500 ms produces no action the firmware detects the simultaneous HIGH state, waits 500 ms to classify the gesture, finds at least one sensor has released, and ignores the event. Holding both sensors simultaneously for 500 ms or longer triggers command transmission: after the 500 ms wait the firmware re-reads both pins and, finding both still HIGH, transmits the localBuffer over TCP and waits for release before continuing. The fifth gesture holding both sensors for two full seconds while in emergency mode sends the ACK confirmation to the Appliance Module. Holding only one sensor for two seconds while in emergency mode sends the STOP false-alarm dismissal.

This five-gesture design was deliberately economical. Every gesture is distinguishable by three orthogonal signals: which sensor or sensors are touched, how long they are held, and whether the module is in emergency mode. No two gestures share all three attributes. The 2-second hold threshold for ACK and STOP was chosen specifically to be impossible to trigger accidentally during normal command entry, a distinction that was verified by extensive testing before finalising.

4.2.2.3 Emergency Mode Behaviour in Firmware

When the emergencyMode flag is true, the main loop returns early after the emergency handling block and never reaches the normal input-processing code below it. Inside the emergency block, the firmware fires a 400 ms on / 600 ms off beep-and-vibration cycle at one second intervals using millis()-based timing. On each loop pass it reads both touch sensors. If either is touched, it silences the alert, then blocks for a 2-second window while continuously checking both sensors. After 2 seconds, if both sensors are still held, it transmits "ACK\n" over TCP and clears the emergencyMode flag. If only one sensor remains held after 2 seconds, it transmits "STOP\n" and clears the flag. If neither sensor is held after 2 seconds, meaning the user touched accidentally and withdrew the emergencyMode flag is left true, the emergency timer is reset, and the beep cycle resumes. This design ensures the user cannot accidentally dismiss a real emergency alert.

4.2.3 Buzzer and Vibration Motor Feedback Subcircuit

4.2.3.1 Hardware

A passive buzzer is connected to GPIO 25 and a vibration (coin) motor to GPIO 12. Neither device is driven directly from the GPIO pin both are switched through NPN transistor-based driver circuits in which the ESP32 GPIO drives the transistor base, the transistor collector drives the device, and the emitter connects to ground. This arrangement is necessary because both devices draw currents that exceed the ESP32's safe GPIO source limit of approximately 12 mA; the transistor provides the current gain to switch the load reliably. A flyback diode across the vibration motor suppresses the inductive voltage spike when the motor is switched off, protecting the transistor from breakdown. Both devices activate and deactivate together in all firmware calls, providing simultaneous audio and tactile confirmation on every event.

4.2.3.2 Feedback Pattern Encoding

The triggerFeedback() function implements four of the five coded feedback patterns. An 'S' byte produces a single 400 ms combined buzzer-and-vibration pulse, telling the user that their command executed successfully they may proceed with the next action. An 'E' byte produces a single 800 ms pulse, distinctly longer than 'S', telling the user that the Appliance Module did not recognise the command string they should re-enter. A 'C' byte fires five consecutive 400 ms pulses with 200 ms silent gaps between them, communicating that the password was accepted and the door has unlocked. A 'W' byte fires five 800 ms pulses with 200 ms gaps, communicating password rejection. The distinction between C and W is therefore the duration of each pulse in the five-pulse sequence 400 ms for correct, 800 ms for wrong making them perceptually unambiguous even in a noisy environment.

The fifth feedback condition the 'F' fire alert is handled not by triggerFeedback() but directly inside the emergency mode block of loop(). While emergencyMode is true, the loop fires a 400 ms on / 600 ms off cycle at one-second intervals. This pattern is continuous and repeating rather than a fixed-count sequence, clearly distinguishing it from the five-pulse C and W patterns and from the single S and E pulses. The continuous repetition persists until the user sends ACK or STOP, at which point emergencyMode is cleared and the loop resumes normal operation.

Table 4.1: Feedback Pattern Encoding Buzzer and Vibration Motor

Code	Duration	Meaning
S	Single ~400 ms pulse	Command executed proceed normally
E	Single ~800 ms pulse	Pattern not recognised; re-enter
C	Five short pulses	Password accepted; door unlocks
W	Five long pulses	Password rejected; try again
F	Continuous short pulses	Fire alert active; dual-hold to confirm

Both sensors have triggered simultaneously. User must respond: hold both touch sensors 2 s to send ACK (real emergency) or hold one sensor 2 s to send STOP (false alarm).Buzzer and vibration motor activate simultaneously for every pattern. All durations are firmware-controlled via blocking delays inside triggerFeedback() and the emergencyMode loop block.

4.2.4 Wi-Fi Station Link and TCP Client Connection

The Keypad Module operates exclusively as a Wi-Fi station (WIFI_STA mode), connecting to the access point SSID 'ESP32-Server', password '12345678' broadcast by the Appliance Control Module's ESP32. This design means the TCP link between the two boards is entirely independent of the home router: even if the home router is offline, the two modules remain connected through the ESP32 soft-AP link and all Braille commands continue executing without interruption. The keypad never connects to the home router directly; it reaches the internet through the Appliance Module if at all.

Connection management runs at the very start of every loop() iteration inside maintainConnection(). It first calls WiFi.status(): if the link is not WL_CONNECTED, it calls WiFi.disconnect() and WiFi.begin() to restart the association process, then returns immediately without executing the rest of the function or the main loop body. If Wi-Fi is confirmed connected, it checks tcpClient.connected(): if the TCP socket has dropped, it calls tcpClient.connect(serverIP, 7890) to re-establish. Both recovery actions return immediately rather than blocking, so a failed reconnection attempt does not freeze the loop it will simply try again on the next pass, typically within milliseconds.

4.3 Appliance Control Module

The Appliance Control Module is the power-control and cloud-connectivity half of the system. It runs on a second ESP32-WROOM-32 operating simultaneously as a Wi-Fi soft access point (for the Keypad Module) and a router-connected station (for Blynk cloud). Its responsibilities are to host the TCP server, decode incoming command strings and drive the relay board accordingly, manage the servo door lock, monitor both fire sensors, persist appliance states through power cycles, maintain the Blynk cloud dashboard, and provide local visual feedback through the OLED display. All of these functions run cooperatively in a single non-blocking loop with no task scheduler.

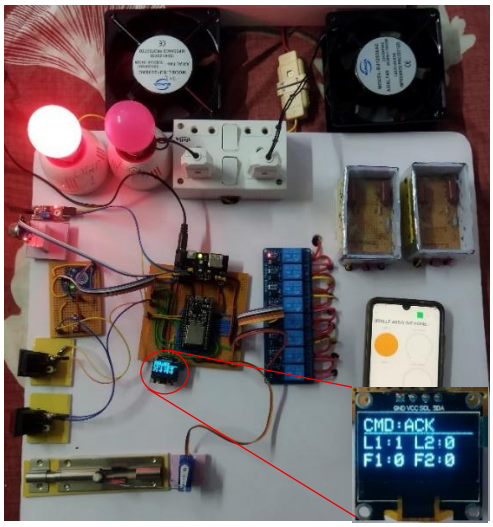

Figure 4.3: The Appliance Control Module prototype showing the relay board, connected room lights and fans, servo-operated door lock, IR flame sensor, MQ-2 gas sensor, and OLED status display.

4.3.1 TCP Server and Command Parsing

4.3.1.1 Server Initialisation and Client Management

A WiFiServer instance is started on port 7890 inside setup() after the Wi-Fi interface is configured. The server accepts exactly one simultaneous client connection. On every loop pass, the firmware calls tcpServer.hasClient(): if a new connection request arrives while an existing active client is already connected, the new client is immediately stopped and discarded, enforcing single-occupancy. If no active client exists, the incoming connection is accepted and stored in the activeClient variable. This policy prevents two simultaneous TCP clients from issuing conflicting commands and keeps the state machine simple.

4.3.1.2 Command Reception and Parsing

While the active client is connected and has bytes available, the firmware reads characters one at a time from the TCP stream and appends them to inputBuffer. When a newline character '\n' arrives, the buffer is trimmed of whitespace and the complete command string is passed to the parsing block. The parser checks inputBuffer against each known command using equality tests and the startsWith() method. An unrecognised string leaves the response variable at its initial value of 'E'; any recognised command sets it to 'S' (or 'C' / 'W' for password responses). The single response byte is written back to the active client with activeClient.print(response) immediately after parsing, before any other loop activity, so the keypad receives its feedback within one TCP round-trip of sending the command. The inputBuffer is then cleared to an empty string ready for the next command.

The complete recognised command vocabulary is: L1 and L2 toggle room lights 1 and 2 individually; Fxy (where x is 1 or 2 and y is 0–4) sets a fan to a specific speed step; ALL1 switches both lights on and drives both fans to maximum in a single command; ALL0 de-energises all relays at once; OPN and CLS control the door lock; ACK confirms a fire emergency as real; and STOP dismisses an alert as a false alarm. Every other string returns 'E' without executing anything, preventing unintended relay state changes from malformed or partially transmitted commands.

Table 4.2. Supported Command Set with Executed Actions

Command	Action Executed
L1 / L2	Toggle Room 1 or Room 2 light; state persists across power cycles
Fxy	Set fan x (1 or 2) to speed step y; 0 switches off, 4 is maximum
ALL1	Switch both lights on and drive both fans to maximum simultaneously
ALL0	De-energise all relays in one command; fastest way to clear everything
OPN	Rotate servo to 180°; auto-return to locked at five seconds
CLS	Return servo to 0° immediately; overrides auto-relock timer
ACK	Confirm fire emergency is real; triggers Blynk alert and opens door

STOP	Dismiss detected alert as false alarm; no caregiver notification sent

4.3.2 Room Light Control Subcircuit

Room Light 1 is driven by a single relay whose coil is connected to GPIO 13, and Room Light 2 by a relay on GPIO 12. Both relays are on the eight-channel relay board, which operates in active-low mode: a LOW signal from the GPIO energises the relay coil and closes the Normally Open contact, connecting the lamp circuit; a HIGH signal de-energises the coil and the contact returns to open, disconnecting the lamp. The macros RELAY_ON and RELAY_OFF are defined as LOW and HIGH respectively to make the firmware intent readable. In setup(), both relay pins are driven HIGH (RELAY_OFF) before any other initialisation, so both lights are unconditionally off at startup regardless of NVS state; the persisted state is applied only after the NVS read completes later in setup().

The L1 and L2 command handlers each invert the corresponding boolean state variable (l1State, l2State), write the new state to the relay pin using the ternary expression digitalWrite(RELAY_L1, l1State ? RELAY_ON : RELAY_OFF), persist the new state to NVS using preferences.putBool(), and update the corresponding Blynk virtual pin (V1 for Light 1, V2 for Light 2). The NVS persistence means that the light states survive a power cycle: on the next startup, setup() reads l1State and l2State back from NVS and applies them to the relay pins, so the room returns to exactly the state it was in before power was lost.

4.3.3 Capacitor-Based Fan Speed Regulation circuit

4.3.3.1 Hardware Principle

Single-phase AC induction motors the type used in standard ceiling or table fans cannot be effectively speed-controlled by simply reducing the supply voltage, because torque drops non-linearly and the motor may stall. The technique used here inserts a capacitor in series with the motor winding: the capacitor's reactance ($1 / 2\pi fC$) limits the current through the winding without resistive power dissipation, and increasing the capacitance reduces the reactance, increasing the current and therefore the speed. By switching different capacitors in and out of the motor circuit using relays, four distinct speed steps are achievable from a fixed 230 V AC supply. The figure 4.4 shown below describe the circuit.

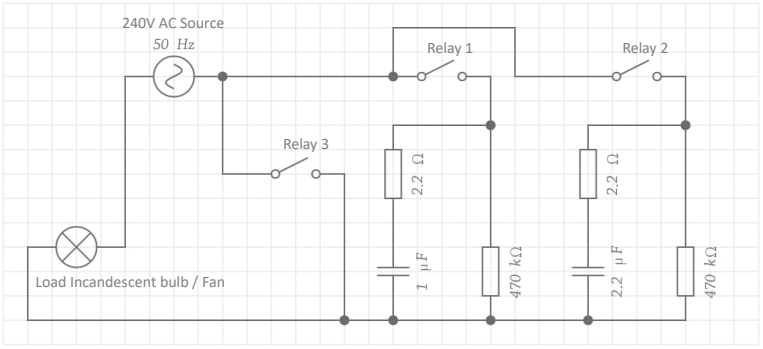

Figure 4.4: The schematic of Capacitor-Based Fan Speed Regulation circuit

4.3.3.2 Relay Allocation and Capacitance Mapping

Three relays are allocated to each fan. Fan 1 uses GPIO 14 (Relay F1_1), GPIO 27 (Relay F1_2), and GPIO 26 (Relay F1_3). Fan 2 uses GPIO 2 (Relay F2_1), GPIO 15 (Relay F2_2), and GPIO 32 (Relay F2_3). Speed 0 de-energises all three relays, disconnecting the motor entirely. Speed 1 (Low) energises Relay 1 only, inserting a 1.0 µF capacitor in series producing barely visible blade rotation. Speed 2 (Low-Medium) energises Relay 2 only, inserting a 2.2 µF capacitor producing a perceptible breeze at arm's length. Speed 3 (Medium-High) energises both Relay 1 and Relay 2 simultaneously; since the two capacitors are now in parallel their capacitances add to 3.2 µF, increasing motor current further for comfortable continuous-occupancy airflow. Speed 4 (Maximum) energises Relay 3 alone, which bypasses all capacitors and connects the full 230 V directly across the motor windings.

Table 4.3: Capacitor-Based Fan Speed Regulation Relay and Capacitance Mapping

Speed level	Relays	Cap (µF)	Observed Behaviour
0 Off	None	0	Coils de-energised; fan stationary
1 Low	Relay 1	1.0	Blade rotation visible, negligible airflow
2 Low-Med	Relay 2	2.2	Perceptible breeze at arm length
3 Med-High	Relays 1&2	3.2	Comfortable for continuous occupancy
4 Maximum	Relay 3	Bypass	Full 230 V across motor windings

Commands apply identically to Fan 1 (x=1) and Fan 2 (x=2). Speed 3 is the only state where two relays are simultaneously energised, producing the parallel capacitor combination of 3.2 µF.

4.3.3.3 Firmware Implementation

The setFan1Speed() and setFan2Speed() functions implement the relay combination logic using conditional expressions. Relay 1 is set to RELAY_ON when speed == 1 || speed == 3, Relay 2 is set to RELAY_ON when speed == 2 || speed == 3, and Relay 3 is set to RELAY_ON only when speed == 4. All three relays are driven to RELAY_OFF for speed 0. This means speed 3 is the only state where two relays are energised simultaneously, creating the parallel capacitor combination. After setting the relay pins, each function calls Blynk.virtualWrite(V3, speed) or Blynk.virtualWrite(V4, speed) to update the Blynk dashboard gauge. Fan states are persisted to NVS with preferences.putInt() and restored on startup identically to the light states. Fan command 'Fxy' extracts the fan number from character index 1 and the speed digit from character index 2 of the command string using charAt() - '0' integer arithmetic, then calls the appropriate speed function.

Manual fan speed buttons on GPIOs 34 and 35 provide a local hardware override. A short press on a fan button toggles the fan between off and its last non-zero speed using a savedFan variable. A hold longer than 500 ms enters a continuous cycling mode that increments the speed from 1 through 4 and back to 1 in 500 ms steps for as long as the button is held. This behaviour is implemented using two millis()-based timestamps per fan: one to detect when the 500 ms hold threshold is crossed (entering long-press mode), and one to pace the 500 ms step intervals within the long-press cycle.

4.3.4 Servo Motor Door Lock

An SG90 micro-servo is connected to GPIO 25 through the ESP32Servo library. The servo is initialised to 0° in setup(), representing the locked position. The OPN command writes 180° to the servo using doorServo.write(180), physically rotating the locking bolt to the open position, and simultaneously records the current time in doorOpenTime and sets the doorAutoClose flag to true. At the very beginning of every subsequent loop() iteration checked before any TCP or sensor processing the firmware tests whether doorAutoClose is true and whether millis() - doorOpenTime has exceeded 5000 ms. If both conditions are met, it writes 0° to the servo and clears the doorAutoClose flag, re-locking the door without any further user action. The CLS command overrides this timer at any time by writing 0° immediately and setting doorAutoClose to false.

The 5-second auto-relock window was determined empirically by observing users walking through the door during prototype testing. Three seconds felt rushed when hands were

full; ten seconds left the door visibly unsecured for long periods. Five seconds balanced convenience against security and measured consistently at 5.0 ± 0.1 s across twenty timed trials. When the ACK emergency confirmation arrives, the door opens to 90° rather than the full 180° of the normal OPN command partial opening is sufficient for emergency egress while limiting exposure and the auto-close timer is started simultaneously, ensuring the door always re-locks.

4.3.5 Dual-Sensor Fire Detection circuit

The IR flame sensor and MQ-2 combustible gas sensor which are shown in Fig 11 & Fig 10 . Both must trigger simultaneously for any alert to be generated, eliminating all single-sensor false positives.

4.3.5.1 Sensor Hardware and Polarity

The IR flame sensor is wired to GPIO 4 and operates active-HIGH: it outputs a logic HIGH when it detects infrared radiation in the spectral band emitted by open flames, and returns to LOW otherwise. The MQ-2 combustible gas sensor is wired to GPIO 17 and operates active-LOW: it normally outputs HIGH and transitions to LOW when combustible gas concentration exceeds its detection threshold. In the firmware, these polarities are correctly handled by the expressions flame_detected = (digitalRead(FLAME_PIN) == HIGH) and gas_detected = (digitalRead(MQ2_PIN) == LOW). Reading both on every single loop pass with no sampling interval means the detection latency is bounded only by the loop execution time, which is sub-millisecond when no long operations are pending.

4.3.5.2 Dual-Trigger Requirement and False-Alarm Suppression

The critical design rule is that neither sensor can generate an alert alone both must cross their respective thresholds simultaneously. The firmware enforces this with a single AND condition: if (gas_detected && flame_detected). This requirement came directly from a failure observed during prototype testing: a standalone IR flame sensor in a south-facing room triggered repeatedly at sunset because ambient daylight alone saturated its photodetector. Afternoon sunlight does not simultaneously produce combustible gas concentrations above the MQ-2 threshold, so the AND condition cleanly separates genuine fire events from environmental interference. Across all single-sensor test scenarios IR triggered alone by a desk lamp, gas triggered alone by nearby cooking activity zero alerts were generated.

When both conditions are simultaneously true, the firmware sets fireActive to true, updates the OLED to display '!! FIRE !!', and begins sending the single-byte character 'F' over the TCP connection to the Keypad Module. The sending is rate-limited to once per second

using the lastFireMsg timestamp to avoid flooding the TCP receive buffer. The Keypad Module receives 'F', sets its emergencyMode flag, and begins the continuous alert beep cycle. The Appliance Module continues sending 'F' on every second until it receives either 'ACK\n' or 'STOP\n' from the keypad. On ACK, it clears fireActive, opens the door, and dispatches the Blynk fire_alert notification. On STOP, it clears fireActive and returns to normal operation with no Blynk notification sent.

4.3.5.3 MQ-2 Warm-Up Limitation

The MQ-2 sensor requires a 30-second warm-up period after power-on during which its internal heater reaches operating temperature and its sensitivity stabilises. During this window, the sensor output is unreliable and may not correctly cross its threshold even in the presence of combustible gas. A fire event occurring in the first 30 seconds after system startup might therefore not trigger the gas threshold condition, meaning the dual-sensor requirement cannot be fully satisfied and no alert would be generated even if flame is present. This is documented as a known hardware limitation of the MQ-2 sensor itself and is not addressable in firmware without an external gas reference.

4.3.6 Hardware Password Entry Circuit

The password circuit uses two components to encode a binary credential sequence and submit it for verification. A dedicated pushbutton (BTN_PASS) on GPIO 5 serves as the data-bit input, wired with INPUT_PULLUP so it reads HIGH when unpressed and LOW when pressed. A TTP223 capacitive touch sensor (TOUCH_PASS) on GPIO 23 serves as the combined clock and submit trigger.

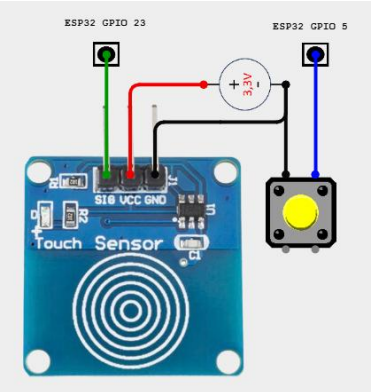

Figure 4.5:Password entry circuit using touch sensor and push-button

To enter each bit of the password, the user sets the data button to the desired bit value held pressed for a '1', released for a '0' and then briefly taps the touch sensor. The firmware

detects the touch sensor's falling edge: when touchHandled is true and the touch sensor transitions from HIGH to LOW after having been held for less than one second, it reads the current state of BTN_PASS. If BTN_PASS is LOW the appended bit is '1'; if HIGH the appended bit is '0'. The resolved bit character is appended to passInput and the OLED updates to show the growing password string prefixed with 'PW: '.

When the touch sensor is held for more than two seconds detected by the condition millis() - touchStartTime > 2000 while touchHandled remains true the firmware interprets this as the submit gesture. It compares passInput against the CORRECT_PASSWORD constant (default: "1010"). If they match, 'Access GRANTED' is displayed on the OLED and the 'C' byte is sent to the Keypad Module over TCP, triggering the five-short-beep confirmation pattern. If they do not match, 'Access DENIED' is displayed and the 'W' byte is sent, triggering the five-long-beep rejection pattern. In both cases passInput is cleared to empty after evaluation. Holding BTN_PASS pressed for more than two seconds at any point clears passInput immediately, providing a mid-entry reset without requiring the user to submit an incorrect entry first.

4.3.7 Manual Override Switches

Two manual toggle switches MAN_SW1 on GPIO 18 and MAN_SW2 on GPIO 19, both configured as INPUT_PULLUP provide direct physical control of the two room lights for sighted caregivers or others who do not use the Braille keypad. On every loop pass, the firmware reads the current state of each switch using digitalRead() and compares it against the saved previous state (lastSw1State and lastSw2State). Any change in state regardless of whether the switch moved from HIGH to LOW or LOW to HIGH triggers the corresponding light toggle: the boolean state is inverted, the relay pin is updated, the new state is persisted to NVS, and the Blynk virtual pin is updated. This edge-detection approach means every physical flip of the toggle switch produces exactly one light state change, matching the user's intuitive expectation.

4.3.8 OLED Status Display

A 128×64 pixel SSD1306 monochrome OLED display connects to the ESP32 via I²C on GPIO 21 (SDA) and GPIO 22 (SCL), driven by the Adafruit_SSD1306 library. The display is probed at both common I²C addresses 0x3C and 0x3D during setup(); if found at either address the oledStatus flag is set to true, and all subsequent display calls are guarded by if(oledStatus), so the module operates identically whether or not the display is physically

present. The display is intended for sighted caregivers and technicians rather than the primary user.

The showStatus() function renders a structured three-line readout at 2× text scale. The first line shows the label 'CMD:' followed by the last executed command string, truncated to five characters if longer. A horizontal white separator line follows. The second line shows 'L1:' and 'L2:' followed by their boolean states (0 or 1). The third line shows 'F1:' and 'F2:' followed by their integer speed values (0 through 4). Time-critical one-off messages 'Door Open', 'Door Closed', 'Access GRANTED', 'Access DENIED', '!! FIRE !!', 'Alert Confirmed', 'False Alarm', and 'Input CLEARED' are rendered by updateOled(), which clears the screen and writes the message at 2× scale for maximum visibility, then returns to the showStatus() layout after a fixed delay.

4.3.9 Blynk Cloud Integration and State Synchronisation

Blynk is initialised in setup() using Blynk.config() with the template ID, template name, and authentication token that bind the device to the 'BRAILLE ASSISTIVE HOME' cloud dashboard. Blynk.connect() is called only if the router Wi-Fi connection succeeded; if the router is unreachable, the module continues operating locally without cloud connectivity, with all relay control and fire detection functioning normally. Blynk.run() is called at the start of every loop() iteration to service the Blynk heartbeat, process any cloud-initiated callbacks, and keep the persistent MQTT-like connection alive. Every call to virtualWrite() is guarded by Blynk.connected() to prevent firmware hangs if the cloud link drops mid-session.

Five virtual pins map the appliance states to dashboard widgets. V1 and V2 are boolean light state indicators updated by the L1, L2, ALL1, and ALL0 handlers. V3 and V4 carry the integer fan speed values (0–4) and are updated by every setFan1Speed() and setFan2Speed() call, including those triggered by manual fan buttons. The dashboard therefore always reflects the live state of every appliance regardless of whether the command came from the Braille keypad, a manual switch, or a hardware button. Emergency fire notifications are dispatched via Blynk.logEvent('fire_alert', ...) exclusively from the ACK handler never automatically on sensor trigger. This gate ensures that every notification the caregiver receives represents an event that the physically present user has already verified as real, making it far more actionable and reducing alert fatigue.

4.3.10 Non-Volatile State Persistence

The Preferences library is used to read and write appliance states to the ESP32's internal NVS (Non-Volatile Storage) flash partition, keyed under the namespace 'home_auto'. Light states are stored as booleans under keys 'l1' and 'l2'; fan speeds are stored as integers under keys 'f1' and 'f2'. During setup(), the four values are read back from NVS before the relay pins are driven, so the appliances are restored to their pre-power-cycle configuration within milliseconds of startup. Every command handler that changes a light or fan state calls the corresponding preferences.putBool() or preferences.putInt() immediately after updating the relay, so the persisted state is always in sync with the live hardware state. The NVS namespace is opened in read-write mode (false as the second argument to preferences.begin()) so that write operations succeed; a read-only open would silently discard all putBool/putInt calls.

CHAPTER 5

RESULTS

5.0 Introduction

This chapter presents the experimental results obtained from testing the hardware prototype in real-world scenarios. It documents the successful execution of appliance commands, such as light toggling and capacitive-based fan speed adjustments. Furthermore, this section validates the integration of the Blynk IoT platform, confirming that system states and emergency alerts are accurately reflected on the remote dashboard.

5.1 Appliance control results

5.1.1 Light 1 Response for Command L1:

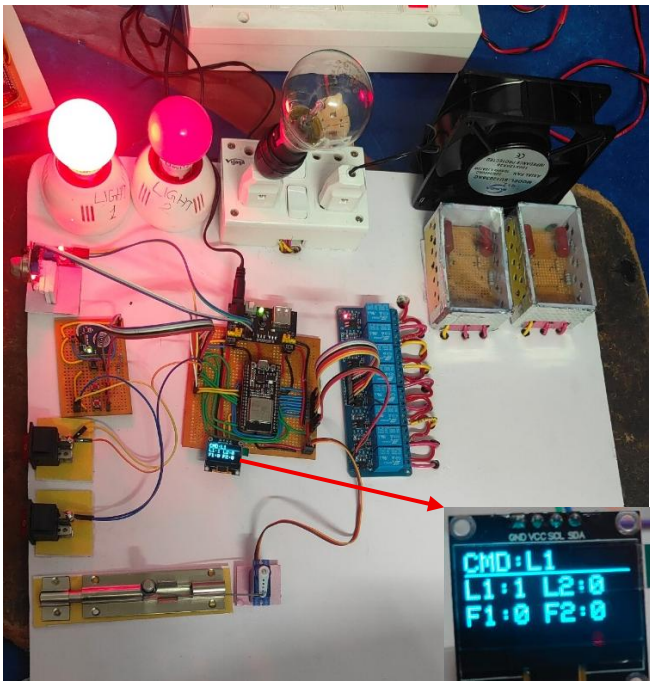

Figure 5.1: Appliance control module response for command L1

Up on givinig the input L1 input by the pressing button 1,2,3 & touching right touch sensor for typing L and button 1 & touching left touch sensor for typing 1, then entered command by a short touch on both sesnors, then the light 1 state toggled to on state i.e. there is a transition from OFF to ON.

5.1.2 Fan 1 response for Command F11:

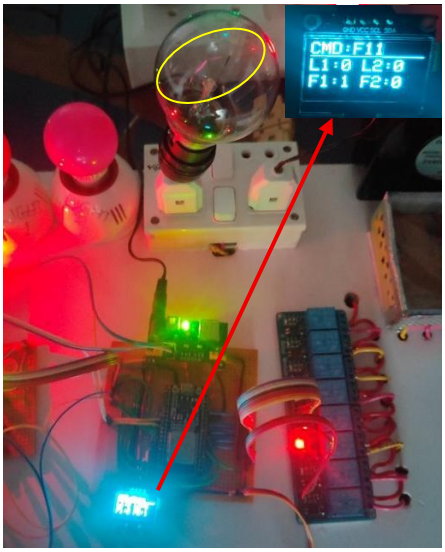

Figure 5.2: Hardware output showing fan1 at regulation level 1.

Up on givinig the input F11 input by the pressing button 1,2,4 & touching right touch sensor for typing F and button 1 & touching left touch sensor for typing 1 for two times, then entered command by a short touch on both sesnors, then the relay connected to high reactance rc channel of regulator, as it is difficult to display air breeze on photo I have used incandesecnt bulb as it show brightness change for each regulation level.

5.1.3 Fan 1 response for Command F12:

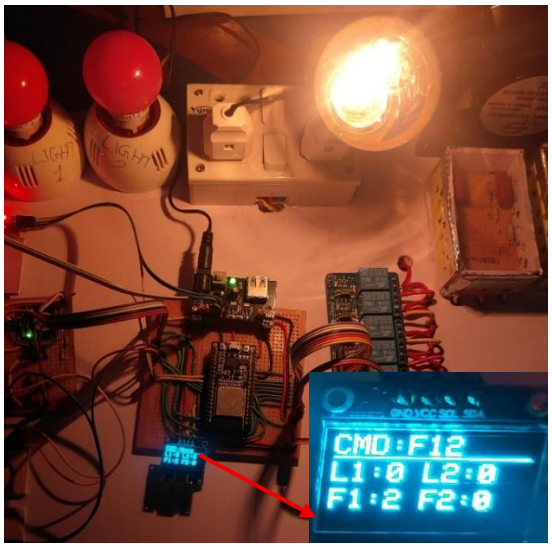

Figure 5.3: Hardware output showing setting fan1 to regulation level 2.

Up on givinig the input F11 input by the pressing button 1,2,4 & touching right touch sensor for typing F , button 1 & touching left touch sensor for typing 1 & button 1,2 & touching left touch sensor for typing 2, then entered command by a short touch on both sesnors, then the relay connected to high reactance rc channel of regulator, as it is difficult to display air breeze on photo I have used incandesecnt bulb as it show brightness change for each regulation level.We can observe the highest brightness at level 4 as it bypass the RC networks.

5.2 Fire monitoring and IoT Integration Results

5.2.1 Fire monitoring results

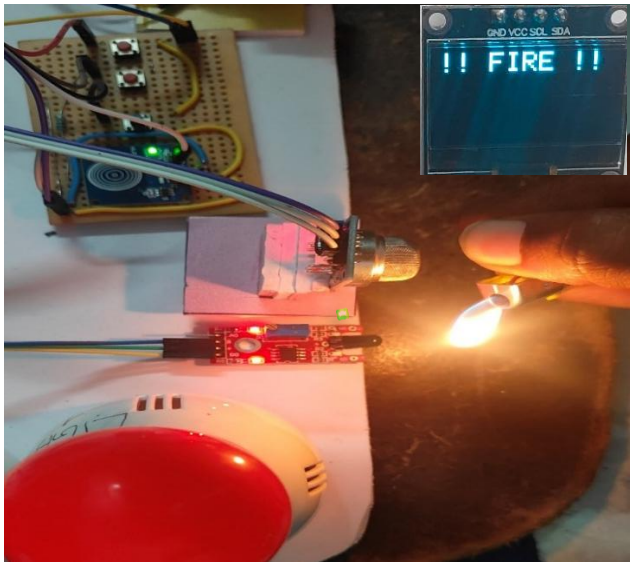

Figure 5.4: Stimulating fire monitoring system

To simulate a high-reliability fire monitoring system, the MQ-2 gas sensor was triggered using smoke to detect combustible particles, while a cigar lighter provided the infrared (IR) signature needed to activate the flame sensor.

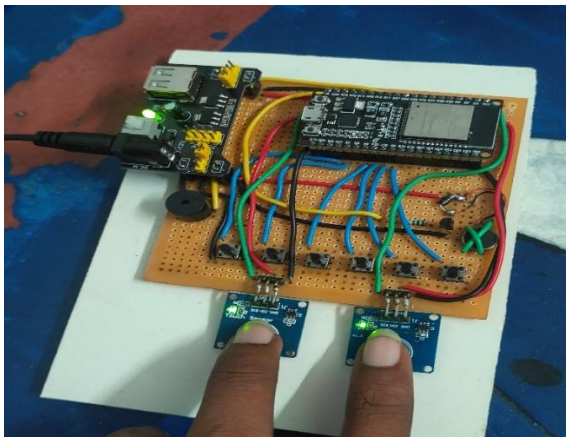

Figure 5.5: sending Acknowledgment(ACK) signal

The system utilizes a 2-second dual-touch hold on the TTP223 capacitive sensors to transfer the ACK command. The ACK command specifies that user confirmed the fire is unintentional & he is in danger and need to notify the care taker.

Figure 5.6: Push notification on the Blynk IoT app

The appliance control module processed the ACK(acknowledgement) command and delivered a notification to care taker via Blynk IoT app.

5.2.2 IoT Integration Results

5.2.2.1 light 1 status monitoring with command L1:

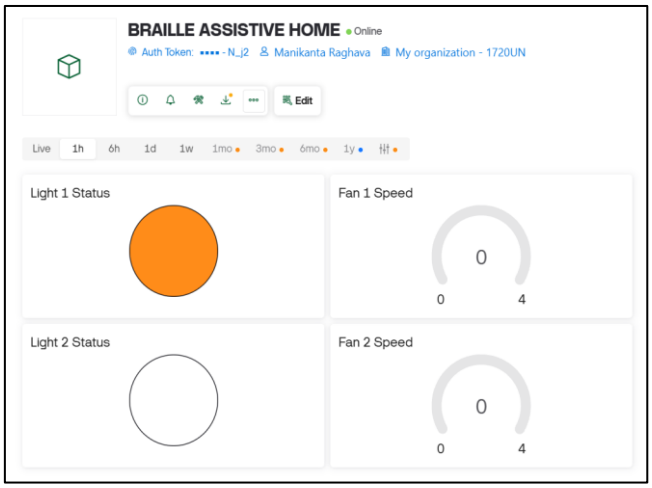

Figure 5.7: Dashboard interface for the Braille Assistive Home IoT project, displaying the real-time status of Light 1

we have monitored the Blynk IOT dashboard during the test hardware prototype. Response of light-1 to command L1 as shown in Fig. 5.1 is completely got reflected on IoT dashboard as the as indicator filled with some colour indicates that light is in ON condition.

5.2.2.2 fan 1 status monitoring with command F11:

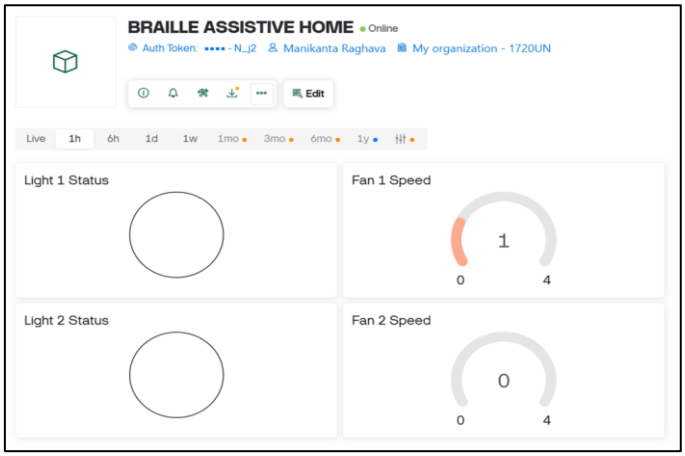

Figure 5.8: Dashboard interface for the Braille Assistive Home IoT project, displaying the real-time status of fan 1 regulation level at level 1

we have monitored the Blynk IOT dashboard during the test hardware prototype. Response of fan 1 to command F11 as shown in Fig. 5.2 is completely got reflected on IoT dashboard synchronizing the regulation level 1 to the gauge value 1 ,indicated through numerical and graphical manner.

5.2.2.3 fan 1 status monitoring with command F12:

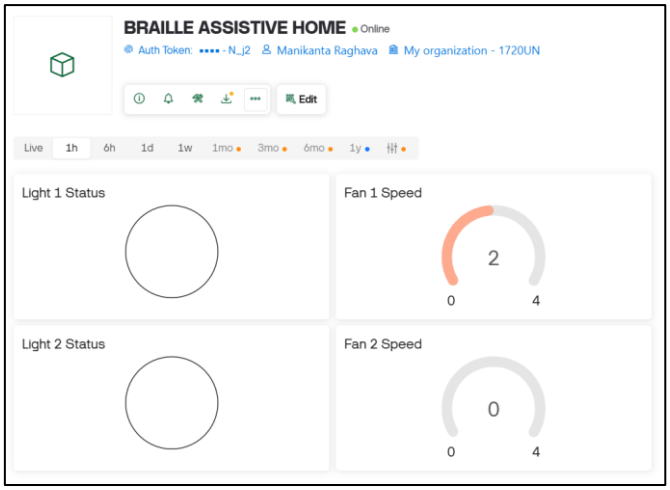

Figure 5.9: Dashboard interface for the Braille Assistive Home IoT project, displaying the real-time status of fan 1 regulation level at level 2

we have monitored the Blynk IOT dashboard during the test hardware prototype. Response of fan 1 to command F12 as shown in Fig. 5.3 is completely got reflected on IoT dashboard synchronizing the regulation level 2 to the gauge value 2, indicated through numerical and graphical manner.

5.3 Password circuit results

5.3.1 Entering correct password

Figure 5.10: OLED display transitions showing (left) the typed password being validated as correct and (right) the resulting "ACCESS GRANTED" message.

5.3.2 Entering incorrect password

Figure 5.11: OLED display transitions showing (left) the typed password being validated as incorrect and (right) the resulting "ACCESS DENIED" error message.

The password circuit the tested by entering both correct and incorrect passwords for them I got the access granted on OLED along with 5 short beeps & vibrations on braille keypad module and access denied on OLED along with 5 long beeps & vibrations on braille keypad module.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.0 Introduction

This chapter concludes the research by summarizing how the tactile-first design and dual-microcontroller architecture provide a reliable, sight-independent automation solution. It highlights the effectiveness of the multi-modal feedback and dual-sensor fire detection in reducing false alarms and alert fatigue. Finally, the chapter outlines a future scope that includes the development of a wearable Braille glove and Braille-to-voice features.

6.1 Conclusion

This project successfully integrates tactile-first design with a robust IoT infrastructure to deliver a fully functional, sight-independent home automation system. By utilizing a six-dot Braille interface, it empowers visually impaired users to manage lighting, fan speed, and door access independently, eliminating the need for visual displays or voice recognition. The dual-microcontroller architecture ensures reliability by separating the Braille input logic from high-voltage relay interference, while a hybrid Wi-Fi configuration maintains core functionality even without an active internet connection.

A sophisticated multi-modal feedback system provides real-time haptic and auditory responses, allowing for unambiguous system confirmation without visual cues. Furthermore, the dual-sensor fire detection protocol eliminates false positives by requiring simultaneous triggers from both flame and gas sensors, while the emergency protocol ensures that notifications sent via the Blynk platform represent verified crises. Ultimately, the system validates that hardware-level inclusive design can effectively restore environmental autonomy and provide a rigorous, reliable solution for users with visual impairments.

6.2 Future Scope

- The system can be further evolved into a compact wearable architecture to enhance portability. This development would allow for a more ergonomic design, making the technology easier for users to carry and operate in mobile environments.
- New Braille-to-voice features will be added to help a wider range of people interact with the system more effectively.

REFERENCES

[1] S. Kucukdermenci, "Raspberry Pi based braille keyboard design with audio output for the visually challenged," Proc. 1st Int. Conf. Modern and Advanced Research (ICMAR), pp. 334–339, 2023.

[2] N. Litayem, "Scalable Smart Home Management with ESP32-S3: A Low-Cost Solution for Accessible Home Automation," Proc. 2024 Int. Conf. Computer and Applications (ICCA), 2024. DOI: 10.1109/ICCA62237.2024.10927887

[3] World Health Organization, World Report on Vision. Geneva: WHO, 2023.

[4] R. R. A. Bourne et al., "Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment," Lancet Global Health, vol. 9, pp. e144–e160, 2021.

[5] M. M. Islam, M. A. Rahaman and M. R. Islam, "Development of Smart Home Technology Based on Internet of Things," SN Comput. Sci., vol. 1, no. 185, 2020.

[6] P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, Protocols and Applications," J. Electrical and Computer Engineering, Article ID 9324035, 2017.

[7] Kumar, A. Singh and R. Kumar, "Voice Recognition Based Home Automation System," Int. J. Engineering Research and Technology, vol. 9, pp. 245–249, 2020.

[8] S. Dhingra et al., "Internet of Things-Based Smart Health Care and Home Automation System," IEEE Internet of Things J., vol. 6, pp. 5577–5584, 2019.

[9] Braille Authority of North America, Unified English Braille Code. Ferrum, VA: BANA, 2019.

[10] N. Patel and S. Shah, "False Positive Analysis of IR-Based Flame Sensors in Residential Environments," Int. J. Electronics and Communication Engineering, vol. 7, pp. 112–118, 2019.

CLASSIFICATION OF PROJECT

	Application	Product	Research	Review
Classification of Project	✓			

PROJECT COs — POs / PSOs MAPPING

Course Title: B.Tech Capstone Project

Course Code: EC3572

Department: Electronics and Communication Engineering

Batch Number: C5

Academic Year: 2025–2026

1. Project Course Outcomes (COs)

CO No.	Course Outcome Statement
CO1	Design and implement a dual-microcontroller IoT home automation system using two ESP32-WROOM-32 boards, integrating a six-button Braille keypad, relay board, servo door lock, OLED display, MQ-2 gas sensor, IR flame detector, and Blynk IoT cloud for sight-independent appliance control.
CO2	Apply a 6-bit pattern-matching algorithm and lookup-table firmware to decode standard Braille cell inputs into ASCII command strings, incorporating dual capacitive touch sensor logic for alphanumeric mode switching and command transmission.
CO3	Develop resilient embedded firmware in C++ implementing TCP client-server communication over a hybrid WIFI_AP_STA network, enabling persistent local appliance control and fire safety monitoring independent of external internet connectivity.
CO4	Implement a multi-sensor dual-trigger safety system combining IR flame detection and MQ-2 gas sensing with a human-in-the-loop emergency acknowledgement protocol, and integrate Blynk IoT cloud notifications for verified, false-alarm-suppressed caregiver alerts.
CO5	Evaluate system performance by validating Braille command recognition accuracy, TCP communication latency, haptic/auditory feedback response times, fire detection reliability, and password security, and document results with hardware schematics, firmware flowcharts, and module integration reports.

2. Program Outcomes (POs)

PO No	Program Outcome Statement
PO1	Engineering Knowledge
PO2	Problem Analysis
PO3	Design / Development of Solutions
PO4	Conduct Investigations of Complex Problems
PO5	Engineering Tool Usage
PO6	The Engineer and the World
PO7	Ethics
PO8	Individual and Collaborative Team Work
PO9	Communication
PO10	Project Management and Finance
PO11	Life Long Learning

3. Program Specific Outcomes (PSOs)

PSO No.	PSO Statement
PSO1	Apply the knowledge of Electronics and Communication Engineering to design, build and test analog and digital circuits, communication systems, and embedded systems for real-world applications.
PSO2	Utilise programming, simulation tools, and hardware platforms to develop innovative solutions addressing societal challenges in healthcare, communication, and automation domains.

4. CO – PO / PSO Mapping Table (3 – High, 2 – Moderate, 1 – Low)

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	2	3	2	3	2	-	2	2	2	2	3	3
CO2	3	3	2	2	3	1	-	2	2	1	2	3	3
CO3	3	2	3	2	3	2	-	2	2	2	2	3	3
CO4	2	2	3	2	3	3	-	2	2	2	3	2	3
CO5	2	3	2	3	3	2	2	2	3	2	2	3	3

5. Justification of CO – PO / PSO Mapping

CO1: CO1 maps strongly to PO1 (engineering knowledge of ESP32, relays, and Braille hardware), PO3 (full dual-microcontroller system design and implementation), PO5 (use of Arduino IDE, Blynk IoT, and TCP/IP stack), PSO1 and PSO2 (embedded assistive system design for automation).

CO2: CO2 aligns with PO1 (firmware logic and Braille encoding theory), PO2 (analysis of six-dot pattern decoding and mode disambiguation), PO5 (implementation of lookup-table algorithms and capacitive touch logic on ESP32), PSO1 and PSO2 (software-hardware integration for accessibility).

CO3: CO3 covers PO3 (TCP client-server firmware architecture), PO5 (WIFI_AP_STA hybrid networking), PO8 (team coordination in distributed embedded development), PSO1 and PSO2 (resilient offline-first system for real-world assistive application).

CO4: CO4 addresses PO3 (safety system design), PO5 (Blynk platform, multi-sensor fusion), PO6 (global impact of assistive IoT for the visually impaired), PO11 (emerging human-in-the-loop IoT safety paradigms), PSO1 and PSO2 (communication and sensor integration for societal benefit).

CO5: CO5 maps to PO2 & PO4 (experimental validation and performance analysis), PO5 (measurement tools and hardware testing), PO7 (ethical design for vulnerable user groups), PO9 (technical documentation and schematic communication), PSO1 and PSO2 (evaluation methodology in ECE assistive technology projects).

6. Sustainable Development Goals (SDG) Alignment

SDG No	Goal	Relevance to the Project
SDG 4	**Quality Education**	Transforms Braille literacy from a classroom skill into a tool for technological empowerment. By utilizing a tactile interface, the project reinforces the practical value of inclusive education and directly supports equal access to specialized skills for persons with disabilities.
SDG 9	**Industry, Innovation and Infrastructure**	Pioneers a hardware-first accessibility platform using a dual ESP32-WROOM-32 topology and Braille-encoded TCP/IP commands. This moves beyond basic software adaptations to create a low-cost, purpose-built IoT infrastructure for inclusive smart homes.
SDG 10	**Reduced Inequalities**	Eliminates the systemic barriers of visual-centric smart home design. By centring the interaction model on the user's existing tactile skills rather than a "sighted paradigm" the project ensures the visually impaired have autonomous, equal access to home automation.

Signature(s) of Students: **Signature of Project Supervisor:**

1.

2.

3.

4.

APPENDIX – A

The QR codes and URLs provide direct access to the complete source code of the developed system modules. These links & QR codes point to the GitHub repository owned by NARAGAM MANIKANTA RAGHAVA bearing the Roll No.23485A0424, which includes the source codes of Braille keypad module and appliance control module. This enables users, evaluators, and developers to easily view, download, and reproduce the project for further study and development.

Braille keypad module QR code & URL:

Figure A.1: QR Code for Accessing Source Code of Braille keypad Module through GitHub Repository.

URL: https://github.com/NMRL24/Braille-Based-IoT-Home-Automation-for-Multi-Disability-Empowerment/blob/main/Braille_keypad_module/Braille_keypad_module.ino

Appliance control module QR code & URL:

Figure A.2: QR Code for Accessing Source Code of Appliance Control Module through GitHub Repository.

URL: https://github.com/NMRL24/Braille-Based-IoT-Home-Automation-for-Multi-Disability-Empowerment/blob/main/appliance_control_module_major_project/appliance_control_module_major_project.ino

APPENDIX – B

The project work has been published as a peer-reviewed research paper in the IJERT journal. The paper explains the system architecture, experimental procedure, and performance results in an organized manner. It contains numerical data, theoretical discussion, and verification of the results. The article can be accessed using the DOI link (https://doi.org/10.5281/zenodo.19185378), which gives a permanent and citable reference for the work. The QR code below takes the reader directly to the IJERT webpage where the paper is hosted, allowing quick access to the full text for study and citation.

Figure B.1: QR Code for Accessing "Braille-Based IoT Home Automation System for Visually Impaired Users using ESP32 Microcontroller" – Published in IJERT, Vol. 15, Issue 03, March 2026

Published by :
https://www.ijert.org/
An International Peer-Reviewed Journal

International Journal of Engineering Research & Technology (IJERT)
ISSN: 2278-0181
Vol. 15 Issue 03 , March - 2026

Braille-Based IoT Home Automation System for Visually Impaired Users using ESP32 Microcontroller

V. Vittal Reddy[1], Naragam Manikanta Raghava[2], Modugumudi Jahnavi[2], Namburu Venkata Siva Prasanth[2] and Mohammed Habeeb Pasha[2]

[1]Associate Professor, [2]Final Year Students - Dept. of Electronics and Communication Engineering
Seshadri Rao Gudlavalleru Engineering College, Andhra Pradesh 521356, India

ABSTRACT - Home automation has become affordable and widespread. It has not become accessible for the visually impaired persons(there are 295 million people live with moderate to severe visual impairment across the world), nearly all encounter immediate barriers with standard interfaces: touchscreens demand sight, voice recognition demands quiet rooms and clear diction. We built for Braille readers instead. Two ESP32-WROOM-32 microcontrollers — one managing a six-pushbutton Braille keypad, one driving an eight-channel relay board — communicate over a persistent TCP/IP link. The keypad module provides tactile and auditory confirmation of every action. The appliance module controls two room lights, two capacitor-regulated fans with four distinct speed steps, a servo-operated door lock, and a dual-sensor fire detection circuit that pairs an IR flame detector with an MQ-2 gas sensor to suppress false alarms. Blynk cloud connectivity allows remote monitoring and caregiver alerts. Commands reached the relay board in under 350 ms under typical home Wi-Fi. Fire alerts dispatched in under 500 ms. The system ran without fault for 48 continuous hours. No false fire alert was generated across all single-sensor test scenarios.

Index Terms - Braille Tactile Interface, IoT Assistive Technology, Visually Impaired Home Control, ESP32, Dual-Sensor Fire Detection, Blynk Cloud Notification

I. INTRODUCTION

Building a connected home has become inexpensive. A thirty-dollar board now handles what required dedicated infrastructure a decade ago. Battery life extends to months, and cloud APIs reduce setup to hours, not weeks. The ecosystem has matured quickly, and yet one fundamental problem has gone almost entirely unaddressed: standard smart home interfaces assume you can see. Every touchscreen dashboard, every voice assistant that mishears commands in a noisy room, every app that requires reading a confirmation dialog — these are barriers that affect approximately 295 million people globally, with roughly 40 million experiencing total blindness [4].

When we surveyed the published literature before starting this project, the pattern was striking. Nearly every accessible smart home proposal either adapted a standard touchscreen interface with screen readers — which transfers, rather than removes, the visual dependency — or relied on voice control, which has documented reliability issues for elderly users and degrades significantly in acoustically difficult environments [7, 8]. We found Braille-based input devices, including Raspberry Pi keyboard designs [1] and newer ESP32-S3 implementations [2], but none of these were integrated systems capable of controlling home appliances. They were input peripherals without a connected purpose.

Louis Braille's 1824 code remains the dominant tactile literacy medium. No competing system has achieved comparable global adoption. Its six-dot cell encodes the Latin alphabet, digits, and punctuation through 64 distinct patterns that millions of users already know. Building a command interface on top of that foundation meant our users needed to learn command abbreviations, not a new physical interaction method. That distinction mattered to us throughout the design.

This paper describes the complete hardware design, firmware architecture, and experimental evaluation of the system we built. Two specific decisions shaped everything else: using a dual-microcontroller topology so the keypad and relay hardware could be physically separated, and requiring simultaneous triggering of both fire sensors before any emergency alert could be generated. Both decisions came directly from limitations we identified in prior implementations.

II. PROPOSED SYSTEM

The central architectural decision was separating user interaction from appliance control. We chose two physically independent ESP32-WROOM-32 microcontrollers connected over a persistent Wi-Fi TCP link, rather than a single board with extended wiring. This let us mount the Braille Keypad Module wherever the user needed it — a bedside table, a wheelchair armrest — and locate the relay hardware near the electrical distribution point. The wiring run between them became a Wi-Fi signal rather than a multi-meter cable bundle.

Initial designs used a single controller. The problem appeared immediately during bench testing: cable length added noise, relay switching caused resets, and debugging one module required physically accessing the other. Splitting the system cost an extra microcontroller; it saved weeks of hardware debugging and produced a more reliable result.

On the user-facing side, a six-pushbutton array maps directly onto the standard Braille cell. Users press dot-pattern combinations to build command abbreviations, then transmit via capacitive touch sensors. Haptic and audio feedback confirm each action within milliseconds. Emergency signalling

International Journal of Engineering Research & Technology (IJERT)
ISSN: 2278-0181
Vol. 15 Issue 03 , March - 2026

uses a two-second simultaneous hold on both sensors — a gesture we tested extensively to confirm it was distinct enough from normal operation that it was never triggered accidentally.

On the appliance-control side, the relay board manages two lights, two fans, and a door lock. A fire detection loop runs in parallel with command processing, monitoring an IR flame sensor and an MQ-2 gas sensor continuously. Neither sensor alone can generate an alert; both must cross threshold simultaneously. Blynk cloud integration provides remote visibility and caregiver push notifications, but only dispatches after the on-site user physically acknowledges the alert.

III. IMPLEMENTATION DIAGRAM

Fig. 1 shows the complete system architecture detailing about the system implementation. The Braille Keypad Module sits at the bottom of the diagram: six pushbuttons feed the left ESP32-WROOM-32, which decodes Braille patterns and sends ASCII command strings over TCP to the Appliance Control Module. That module drives the eight-channel relay board, the servo door lock, and monitors both fire sensors. IoT integration through Blynk adds a cloud layer for remote visibility and alert dispatch to the caregiver mobile app.

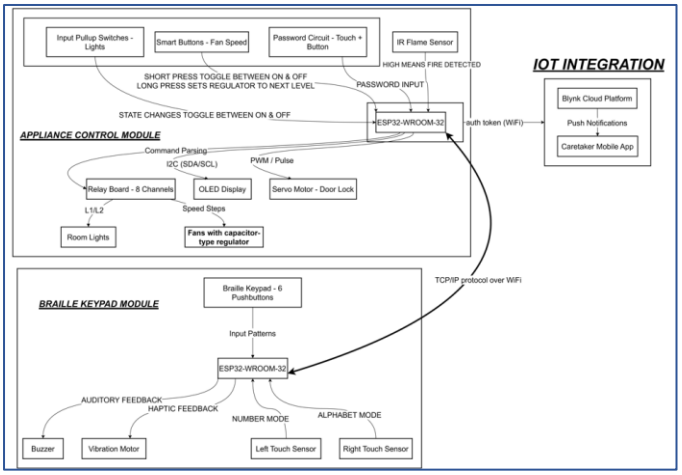

Fig. 1: System Architecture — Dual ESP32 Configuration with TCP/IP Communication, Relay Control and Blynk Integration

IV. FLOW OF PROJECT

Fig. 2 traces execution through both firmware builds. Power-on initialises sensors and Wi-Fi, then both modules enter their main loops. On the keypad side, each iteration reads the six buttons and both touch sensors. Valid Braille input accumulates into a pattern buffer. When a transmit gesture is detected, the completed command string goes out over TCP and the module waits for a single-byte status response — S, E, C, W, or F — which drives the haptic and audio feedback.

On the appliance side, each loop pass checks for incoming TCP commands, polls manual override switches, reads the password circuit, and checks both fire sensors. Commands go through a lookup against the supported command table; unrecognised strings return the E byte without executing anything. Fire detection runs on every pass: if both sensors cross threshold simultaneously, the module enters alert mode and begins sending periodic fire alert codes upstream until an ACK or STOP command arrives. The cooperative non-blocking structure means a 2-second confirmation window slightly reduces responsiveness to concurrent events — a known limitation flagged for a FreeRTOS revision.

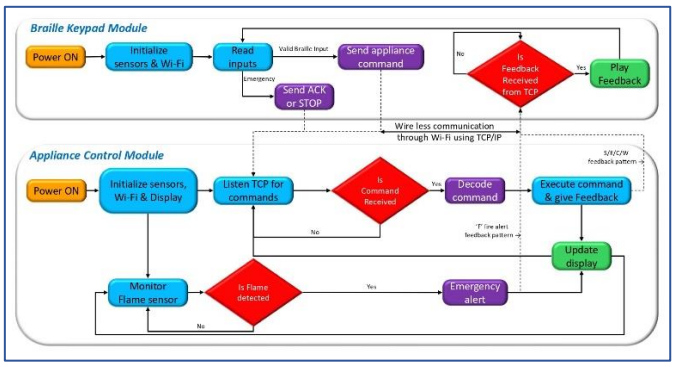

Fig. 2: Firmware Execution Flow for Both ESP32 Modules — Braille Input Decoding, Command Dispatch and Emergency Handling

V. HARDWARE MODULES

A. ESP32-WROOM-32 Microcontroller

We selected Espressif's WROOM-32 module shown in **Fig. 3** for its integrated 2.4 GHz dual-mode radio and dual-core architecture, which eliminates separate wireless coprocessors and keeps the board count manageable. Each module runs in WIFI_AP_STA mode simultaneously — the Appliance Control Module acts as both an access point and a router-connected station. That topology was a deliberate resilience choice: if the home router drops, the two boards maintain their direct access-point link and local appliance control continues without interruption. Only Blynk goes offline; the core functionality survives.

Fig. 3: ESP32-WROOM-32 Microcontroller — Primary Controller for Both Modules

B. Six-Pushbutton Braille Input Array

Each dot position uses a momentary switch tied to internal pull-up resistors, debounced in firmware rather than hardware. We considered hardware RC debouncing in early prototypes; firmware debouncing proved cleaner and allowed us to tune timing for different users without a soldering iron. The six

International Journal of Engineering Research & Technology (IJERT)
ISSN: 2278-0181
Vol. 15 Issue 03 , March - 2026

buttons occupy a row as shown in **Fig. 4**; buttons are arranged in manner of left-to-right mapping braille dots 1 to 6 — any trained reader orients correctly by touch in seconds, without explanation. Accumulated button states form a 6-bit pattern that indexes the encoding table; digit entry distinguishes from letter entry by which touch sensor the user holds, mirroring the Braille Number Indicator convention already familiar to users.

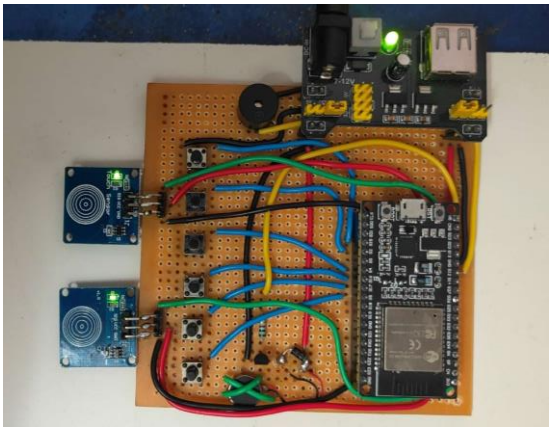

Fig. 4: Braille Keypad Module Prototype — Six-Button Array, TTP223 Touch Sensors, Buzzer and Vibration Motor on Perfboard

C. TTP223 Capacitive Touch Sensors

Two TTP223 sensors are used in the system shown in **Fig. 5**, Left and Right, carry five distinct behaviours between them. Left or Right alone determines the decoding mode — number or letter. Both together for a short tap transmit the buffered command. Both together held for two seconds enter emergency mode. Inside that mode, a two-second dual-hold sends ACK; either sensor held alone for two seconds sends STOP. We achieved five distinct interaction types with two physical components.

Fig. 5: TTP223 Capacitive Touch Sensor — Left and Right Sensors Handle Mode Selection, Transmission and Emergency Control

D. Buzzer and Vibration Motor

As the target user is blind they can easily access the audio feed back to increase accuracy in feed back system also contains a vibration motor driven by a NPN transistor-based driver circuit. The feedback patterns are given in **Table 1**. The buzzer and vibration motor the shown in **Fig. 6**.

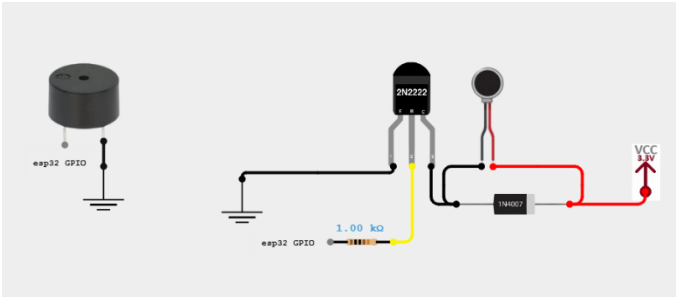

Fig. 6: TTP223 Capacitive Touch Sensor — Left and Right Sensors Handle Mode Selection, Transmission and Emergency Control

E. Eight-Channel Relay Board and Appliance Control

The relay board runs in active-low mode with all contacts wired to Normally Open terminals. We learned the value of that choice during a power interruption test: with contacts wired to Normally Closed, every appliance switched on when power returned. With Normally Open, everything defaults to off on power-up, reset, or unexpected fault. That safe-off behaviour is built into the hardware. No firmware initialisation can override it. The board controls two room lights on two relays, two fans on three relays each for capacitor-based speed regulation (Table 2), and the servo motor door lock. Entire hardware of the appliance control module is shown in **Fig. 7**.

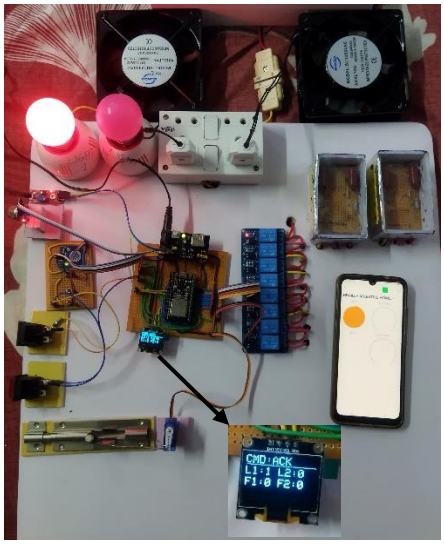

Fig. 7: Appliance Control Module — Eight-Channel Relay Board, Room Lights, Fans, Door Lock, IR Sensor, MQ-2 Sensor and OLED Display

F. Dual-Sensor Fire Detection

We learned the hard way that a standalone IR flame sensor in a south-facing room triggers at sunset. The ambient light alone saturates it. Combining the IR detector with an MQ-2 combustible gas sensor(**Fig. 8**) and requiring both to cross threshold simultaneously addressed this almost completely, because afternoon sun does not simultaneously produce LPG readings. The MQ-2 has a 30-second warm-up after power-on during which its sensitivity is uncalibrated — a fire event in the first half-minute after startup might not trigger the gas threshold, and we document this as a known limitation rather than a solved problem.

International Journal of Engineering Research & Technology (IJERT)
ISSN: 2278-0181
Vol. 15 Issue 03 , March - 2026

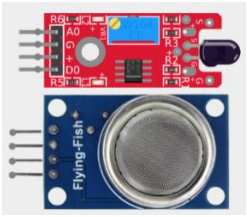

Fig. 8: IR Flame Sensor (left) and MQ-2 Combustible Gas Sensor (right) — Both Must Trigger Simultaneously for an Alert

G. Servo Motor Door Lock

The SG90 (**Fig. 9**) servo on GPIO 25 rotates 180° to unlock and returns to 0° to lock. Automatic re-lock at five seconds came from watching people walk through a door in early testing: hands were full, attention was elsewhere, and nobody sent the explicit CLS command. The five-second delay was a judgement call after trying three and ten seconds with different users. Three felt rushed; ten led to the door sitting open for noticeable periods. Measured auto-relock timing was 5.0 ± 0.1 s across twenty repeated trials.

Fig. 9: SG90 Servo Motor — 180° Rotation Models Door Unlock; Auto-Relocks at 5 Seconds

VI. SOFTWARE TOOLS

A. Arduino IDE and Firmware

Both modules were developed in the Arduino IDE using the ESP32 Arduino core. We chose it over ESP-IDF because the library ecosystem — particularly ESP32Servo, BlynkSimpleEsp32, and the Preferences NVS wrapper — reduced implementation time substantially. The firmware is cooperative non-blocking rather than FreeRTOS task-based. We considered FreeRTOS early in the project; for a prototype where we needed to trace timing interactions between sensor polling and TCP handling, a single traceable loop was easier to debug. The architecture is serviceable but not production-ready — a FreeRTOS migration is the first item in our revision list.

B. Blynk IoT Platform

Blynk's virtual pin model maps cleanly onto appliance states without requiring a custom server. V1 and V2 track light states; V3 and V4 carry fan speed values. Every relay state change updates the corresponding virtual pin synchronously. Emergency notifications go out via the fire_alert logEvent only after the user sends ACK — never automatically. Remote monitoring interface is shown in **Fig. 10** .We made this design decision after reading about caregiver alert fatigue: a notification that the on-site user has physically verified carries more weight than an automatic one, and caregivers are more likely to act on it.

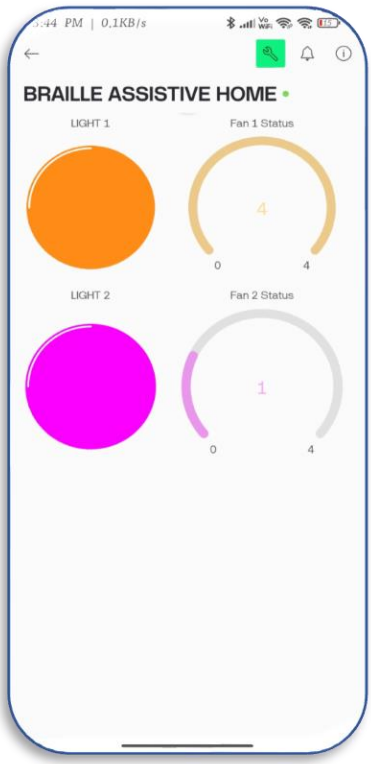

Fig. 10: Blynk 'Braille Assistive Home' Dashboard — Light 1 on, Light 2 on, Fan 1 at Level 4, Fan 2 at Level 1

C. Braille Command Vocabulary

Every function is reachable through a two- or three-character Braille string. L1 and L2 toggle the room lights. Fxy sets fan x to speed y. ALL1 and ALL0 handle global on and off. OPN and CLS control the door. The abbreviations are not standard Braille words — there is memorisation work for new users, and we do not pretend otherwise. In practice, most users in our testing had the command set down after a few sessions. The small vocabulary is a feature: we deliberately chose not to expand it, because every new command adds memory load for blind users who cannot glance at a reference card. These commands are listed in the Table 3.

Table 1. Feedback Pattern Encoding — Vibration Motor and Buzzer Run in Parallel

Code	Duration	Meaning
S	Single ~400 ms pulse	Command executed — proceed normally
E	Single ~800 ms pulse	Pattern not recognised; re-enter
C	Five short pulses	Password accepted; door unlocks
W	Five long pulses	Password rejected; try again
F	Continuous short pulses	Fire alert active; dual-hold to confirm

International Journal of Engineering Research & Technology (IJERT)
ISSN: 2278-0181
Vol. 15 Issue 03 , March - 2026

Table 2. Capacitor-Based Fan Speed Regulation — Measured Relay and Capacitance Mapping

Speed	Relays	Cap (µF)	Observed Behaviour
0 — Off	None	0	Coils de-energised; fan stationary
1 — Low	Relay 1	1.0	Blade rotation visible, negligible airflow
2 — Low-Med	Relay 2	2.2	Perceptible breeze at arm length
3 — Med-High	Relays 1&2	3.2	Comfortable for continuous occupancy
4 — Maximum	Relay 3	Bypass	Full 230 V across motor windings

Table 3. Supported Command Set with Executed Actions

Command	Action Executed
L1 / L2	Toggle Room 1 or Room 2 light; state persists across power cycles
Fxy	Set fan x (1 or 2) to speed step y; 0 switches off, 4 is maximum
ALL1	Switch both lights on and drive both fans to maximum simultaneously
ALL0	De-energise all relays in one command; fastest way to clear everything
OPN	Rotate servo to 180°; auto-return to locked at five seconds
CLS	Return servo to 0° immediately; overrides auto-relock timer
ACK	Confirm fire emergency is real; triggers Blynk alert and opens door
STOP	Dismiss detected alert as false alarm; no caregiver notification sent

VII. RESULTS

We tested what mattered most: speed, fire safety, appliance reliability, and what happens when Wi-Fi drops. Four evaluation areas covered these, though fire detection received disproportionate attention after early prototyping exposed how easily a single IR sensor generates false positives in ordinary room lighting.

A. Command Execution Speed

Most commands hit the relay in under a third of a second. Three hundred fifty milliseconds was our target ceiling, not our average — the distribution clustered well below it under stable Wi-Fi, with occasional outliers when the router was under load. Of that total latency, roughly 20–50 ms was TCP round-trip between the two boards; relay actuation and firmware polling accounted for the rest. What struck us most was the feedback speed: the buzzer and vibration motor fired within ten milliseconds of the response byte arriving, making confirmation feel essentially instantaneous from the user's side.

B. Fire Detection and Alert Dispatch

We tested fire detection with a controlled flame for the IR sensor and a calibrated LPG source for the MQ-2, both triggered simultaneously. Alert codes left the Appliance Control Module within half a second in every trial. The false-alarm suppression result was the one we had been most uncertain about before testing: in every single-sensor scenario — IR alone triggered by a desk lamp, gas alone triggered by cooking activity nearby — no alert state was produced. Getting clean separation between legitimate alerts and sensor noise was the most satisfying engineering result of the project. Blynk notifications arrived on the test phone within roughly three seconds of ACK receipt in all Wi-Fi-connected cases.

C. Appliance Switching and Fan Speed

Light toggle reliability across fifty consecutive Braille-commanded operations: clean transitions every time, no relay bounce, no missed edges. Fan speed testing required tachometric measurement to confirm that rotational velocity actually stepped up at each level — perceptible difference between adjacent speed steps was the practical goal, and we achieved it at all four transitions. Door lock operation was clean across repeated open-close cycles. The servo re-locked at 5.0 ± 0.1 s, measured with a stopwatch app averaged across twenty trials. Password circuit testing through the correct credential and three wrong variants produced the correct C and W feedback patterns in every case.

D. Network Resilience

We cut the router at intervals from ten seconds to two minutes to stress the reconnection logic. Ninety-three percent of disruption events reconnected within ten seconds. The seven failures all occurred when the router itself took longer than expected to complete its own boot sequence — not random failures but a correlated cluster. Once the station interface reconnected, Blynk re-established within five further seconds in every successful case. Throughout every disconnected window, local control never dropped: Braille commands continued executing, relays responded, and fire sensors kept polling. The system also ran fully offline for over 24 hours without any functional loss. All these are mentioned in the **Table 4** shown below.

International Journal of Engineering Research & Technology (IJERT)
ISSN: 2278-0181
Vol. 15 Issue 03 , March - 2026

Table 4. Summary of Measured Performance — All Evaluation Dimensions

Measured Parameter	Result
Braille keypad to relay actuation	Below 350 ms, stable Wi-Fi
Both fire sensors to alert dispatch	Under 500 ms in every trial
ACK receipt to Blynk notification	Approximately 3 s, all cases
Servo auto-relock timing	5.0 ± 0.1 s, 20 trials
Router drop to reconnection	93 of 100 events within 10 s
Unattended continuous operation	48 h with zero faults or resets
False alarms, IR triggered alone	Zero across all single-sensor tests

VIII. CONCLUSION

We set out to build a home automation system that a visually impaired user could operate independently, without a touchscreen and without relying on voice recognition in environments where it often fails. The dual-ESP32 architecture, six-pushbutton Braille keypad, multi-modal feedback circuit, dual-sensor fire detection, and Blynk cloud integration together produced a platform that achieves this. Command latency stayed well within 350 ms. Fire alert generation was comfortably under 500 ms. Forty-eight hours of unattended operation produced no fault, reset, or unexpected state change.

The system has real limitations we document honestly. Braille literacy is required — a reasonable assumption for the target population, but one that excludes newly impaired users. The cooperative firmware loop creates a responsiveness gap during the two-second confirmation window. The MQ-2 sensor is uncalibrated for the first 30 seconds after power-on. These are engineering problems with known solutions; we flag them rather than pretend they do not exist.

The revision list is concrete. FreeRTOS task migration addresses the firmware architecture limitation. A text-to-speech module adds spoken confirmation as an alternative to buzzer patterns. Expanding the command set with natural-language Braille processing removes the memorisation requirement. Longer-term, a wearable Braille input glove enables ambulatory operation. Communication encryption is a prerequisite before any deployment beyond a trusted private network.

IX. REFERENCES

[1] S. Kucukdermenci, "Raspberry Pi based braille keyboard design with audio output for the visually challenged," Proc. 1st Int. Conf. Modern and Advanced Research (ICMAR), pp. 334–339, 2023.

[2] N. Litayem, "Scalable Smart Home Management with ESP32-S3: A Low-Cost Solution for Accessible Home Automation," Proc. 2024 Int. Conf. Computer and Applications (ICCA), 2024. DOI: 10.1109/ICCA62237.2024.10927887

[3] World Health Organization, World Report on Vision. Geneva: WHO, 2023.

[4] R. R. A. Bourne et al., "Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment," Lancet Global Health, vol. 9, pp. e144–e160, 2021.

[5] M. M. Islam, M. A. Rahaman and M. R. Islam, "Development of Smart Home Technology Based on Internet of Things," SN Comput. Sci., vol. 1, no. 185, 2020.

[6] P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, Protocols and Applications," J. Electrical and Computer Engineering, Article ID 9324035, 2017.

[7] A. Kumar, A. Singh and R. Kumar, "Voice Recognition Based Home Automation System," Int. J. Engineering Research and Technology, vol. 9, pp. 245–249, 2020.

[8] S. Dhingra et al., "Internet of Things-Based Smart Health Care and Home Automation System," IEEE Internet of Things J., vol. 6, pp. 5577–5584, 2019.

[9] Braille Authority of North America, Unified English Braille Code. Ferrum, VA: BANA, 2019.

[10] N. Patel and S. Shah, "False Positive Analysis of IR-Based Flame Sensors in Residential Environments," Int. J. Electronics and Communication Engineering, vol. 7, pp. 112–118, 2019.

[11] V. Vujovic and M. Maksimovic, "Raspberry Pi as a Sensor Web Node for Home Automation," Proc. 37th Int. Convention MIPRO, Opatija, Croatia, pp. 1016–1021, 2014.

[12] R. S. H. Istepanian, E. Jovanov and Y. T. Zhang, "Guest Editorial Introduction to the Special Section on M-Health," IEEE Trans. Inf. Technol. Biomed., vol. 8, pp. 405–414, 2004.

[13] S. Kuriakose and R. Kushalnagar, "How blind users perceive the quality of their interactions with mainstream technology," Proc. 21st Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS), pp. 638–640, 2019.

[14] D. Tran, T. Nguyen and H. Pham, "IoT-Based Smart Home System Using Raspberry Pi and ESP32," IEEE Access, vol. 9, pp. 112343–112355, 2021.

[15] J. Gubbi, R. Buyya, S. Marusic and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," Future Generation Computer Systems, vol. 29, pp. 1645–1660, 2013.

[16] Espressif Systems, ESP32 Series Datasheet, version 4.4. Shanghai: Espressif Systems, 2023.